170A WOAC Advanced Powershell Scripting

Site:	CVTE LMS	Printed by:	Brandon Larson,
Course:	170A WOBC/WOIC/WOAC Scripting	Date:	Monday, 27 April 2026, 12:12 PM
Book:	170A WOAC Advanced Powershell Scripting		

Table of contents

Chapter 0: Course Overview

> 💡 **Chapter Purpose**
> This chapter orients you to the course before any content starts. You will learn what the course covers, what the training environment looks like, how the week is structured, what the capstone requires, what is expected of you coming in, and how to get the most out of the five days. Treat this as reference material. You will come back to these pages during the week.

This is a five-day course built around a single problem: taking an operator who already writes PowerShell at a working level and building their ability to produce reliable, remotely-executed, parallelized scripts that can be graded, shared, and reused in the field.

Everything in Chapters 1 through 6 builds toward that capability. The chapter before you right now tells you how the week fits together so every later chapter lands in context.

What is in this introduction

Section	Focus
0.1	What this course is, and what you'll leave with
0.2	The range: Ops and the targets
0.3	The five-day structure
0.4	The capstone, previewed
0.5	Before we start: what you should already know
0.6	How to succeed in this course

> ✅ **How to read this chapter**
> Sections 0.1 and 0.4 are the most important. 0.1 tells you why you are here, and 0.4 tells you what you are working toward. The other four sections are logistics: useful, but lower priority on a first read. Spend 10 minutes on this chapter before Chapter 1.

0.1 - What this course is, and what you'll leave with

Why this matters to an operator: Before you invest five days in this course, you should know exactly what changes in your capability by Friday. This section is the shortest honest answer.

What this course is

170A WOAC Advanced PowerShell Scripting is a five-day course for warrant officers and civilian personnel in cyber-adjacent roles who already write PowerShell at a working level. The goal of the course is not to make you a PowerShell expert. The goal is to move you from "I can write a script on my own machine" to "I can produce tooling that runs reliably across multiple systems, handles failure gracefully, returns structured output, and can be graded against a written contract."

That is a specific transition. It is also the transition that distinguishes an operator who writes scripts for personal use from one whose scripts get reused, shared across a team, and trusted in planning.

What you will leave with

By the end of the course, you should be able to:

- Build a PowerShell function that queries a remote system, handles auth and errors cleanly, and returns structured output another script can consume
- Combine several such functions into a module, and write an orchestrator script that imports the module and drives work across multiple targets in parallel
- Read someone else's PowerShell module, understand what it does, and reason about what will break if you change it
- Diagnose the most common failure modes in remote execution (auth, WinRM, language modes, scope, execution context) using a standard checklist rather than guesswork
- Recognize when PowerShell is the right tool and when it is being abused, from a defender's perspective

What this course does not cover

This is not a red-team course. It is not a threat-hunting course. PowerShell shows up in both of those domains, and Chapter 3.6 touches on the defensive context, but the focus of the five days is operational scripting competence. If you came in expecting offensive tradecraft or adversary emulation, the course will still be useful to you, but the center of gravity is elsewhere.

This is also not a beginner course. Chapter 1 is a deliberate refresh, not a from-scratch introduction. If you have never written a PowerShell function before, you will struggle by Wednesday.

> ✅ **The one-sentence version**
> If someone asks you on Monday what you learned in this course, the answer should be: "I can now build and ship PowerShell tooling that runs reliably across multiple systems and produces output a teammate can actually use." Everything in Chapters 1 through 6 exists to make that sentence true.

0.2 - The range: Ops and the targets

Why this matters to an operator: Every script you write this week runs in a specific environment with a specific layout. Knowing that layout cold (which machine does what, how they talk to each other, what authentication model is in play) is the difference between chasing phantom bugs and diagnosing real ones.

The CVTE range at a glance

The Cyber Virtual Training Environment (CVTE) is a domain-based lab environment hosted on Moodle infrastructure. During this course, you work from a student workstation and drive scripts against four target systems.

System	Role in this course	Notes
Ops	Student workstation, where you run PowerShell and VS Code	Windows 10. PowerShell 7. This is where all scripts originate.
System-1	Remote target	Windows 11. WinRM enabled. Standard CIM/WMI target.
System-2	Remote target	Windows 11. Same role as System-1. Use for parallel testing.
System-3	Remote target	Windows 11. Same role as System-1 and System-2.
Depot	Remote target and report repository	UNC share at `\\Depot\C$\Tmp\` used for output storage later in the course.

Authentication and domain model

The CVTE range is domain-based. All five machines are members of the same Active Directory domain. Authentication uses Kerberos by default, with NTLM as a fallback. This has two practical implications for your scripts:

- When you run remote PowerShell commands from Ops, your current domain credentials are used automatically. You will rarely need to pass `-Credential` explicitly unless you are testing a specific account or simulating an unprivileged caller.
- The double-hop problem (Chapter 3.3) is real in this environment. A command that works from Ops directly may fail when the same command is chained through an intermediate remote session. Chapter 3 explains why and what to do about it.

> 🔧 **Try it (first day, range check)**
> The first thing to do on your Ops workstation is confirm you can reach all four targets. Open PowerShell and run `Test-WSMan System-1`, then repeat for System-2, System-3, and Depot. If any of them return an error, flag it to the instructor before you start Chapter 1. A target that is down on Monday will block your Chapter 5 exercises on Wednesday.

Domain placeholder

Throughout this course, the domain is referred to generically as `<DOMAIN>` or with placeholder names in examples (commonly `CVTELAB`). Your instructor will give you the actual domain name on Day 1. Wherever you see a placeholder in an exercise, substitute the real value from the range you are working on.

0.3 - The five-day structure

Why this matters to an operator: You cannot pace yourself through five days if you do not know where the heavy lifts are. This section tells you which days are intense and which are integration, so you can plan your attention and your evenings.

Week at a glance

Day	Chapters covered	Graded event	Intensity
Monday	Chapter 0 (intro) + Chapter 1 (PowerShell foundations refresh)	Chapter 1 exercise	Light: this is calibration
Tuesday	Chapter 2 (.NET in PowerShell) + Chapter 3 (remoting, security, ops constraints)	Chapter 3 exercise	Medium: new material
Wednesday	Chapter 4 (jobs and parallel execution)	Chapter 4 exercise	Medium: important building block
Thursday	Chapter 5 (orchestration, modules, structured output)	Chapter 5 exercise	Heavy: this is the capstone dress rehearsal
Friday	Chapter 6 (capstone build) + capstone submission	Capstone deliverable	Heavy: work product due

How each day runs

Typical day structure during the course:

- **Morning block (08:00–11:30).** Lecture and walkthrough of the chapter's material. Code examples are demonstrated live. You follow along on Ops.
- **Lunch (11:30–12:30).**
- **Afternoon block (12:30–15:30).** Guided practice and exercises. Instructor is available for questions. This is where most of the learning happens. The morning lecture is scaffolding.
- **Wrap-up (15:30–16:00).** Quick review, questions, any announcements about the next day. Range remains up. Good time to retry anything that did not work in the morning block.

Graded vs ungraded work

Only two things are graded in this course:

1. **Final knowledge assessment** covering 11 knowledge based requirements

2. **The capstone** (Friday). Single submission. Covers four rubric items mapped to the CTSSB skill standard (S1 through S4). This is the majority of your grade.

The end-of-chapter exercises and quizzes in Chapters 1 through 5 are *ungraded formative practice*. You are expected to do them. They are not collected. Their purpose is to prepare you for the capstone, not to generate scores.

> ✅ **Where to invest your attention**
> If you skim any chapter, do not skim Chapter 5. If you skim any exercise, do not skim the Chapter 5 exercise. Chapter 5 is where every pattern from the first four chapters combines into the capstone architecture. Students who struggle on Friday almost always struggled on Thursday.

0.4 - The capstone, previewed

Why this matters to an operator: Every chapter in this book builds toward a single Friday deliverable. Knowing what that deliverable looks like on Monday means every later section has a visible purpose. You are not learning PowerShell concepts in the abstract, you are assembling the pieces of something specific.

The problem the capstone solves

Imagine you have been asked to audit the current state of several systems in your operational area. You need to know what processes are running, what services are installed, and whether a specific monitoring tool has been deployed to each machine. The number of targets is small enough that doing it manually is possible, but large enough that manual work wastes the time of an operator who should be doing analysis, not data collection.

The capstone is the scripted version of that problem. You will build a small PowerShell module and a short orchestrator script that, together, perform the audit against multiple targets in parallel and return structured output ready for review.

What you will submit

Two files, both produced by you:

- **A PowerShell module** containing three functions: two that gather information from remote targets, and one that installs or verifies a tool on those targets. Each function follows the same structured-return pattern: an object with `ComputerName`, `Status`, `Data`, and `Error` properties.
- **An orchestrator script** that imports the module, runs the functions in parallel against a list of targets using PowerShell jobs, collects results, and handles errors at both the job and function level.

You will be given the full written specification on Day 1. Nothing in the submission requirements is held back until Friday.

What you are graded on

Four rubric items, each mapped to one of the four critical skills for this course. The rubric is narrative. You are not being graded on style or elegance, you are being graded on whether your submission demonstrates the four skills cleanly.

The instructor will walk through the rubric in detail during the Chapter 6 material on Thursday afternoon. For now, the point is that the capstone is not a trick. The requirements are fully documented, the patterns are practiced all week, and there are no surprise constraints introduced on Friday morning.

How this book prepares you for it

Every chapter of this book maps to something you will do in the capstone. Specifically:

Chapter	What it contributes to the capstone
1	Function skeleton, structured output, error handling: the shape of every capstone function
2	.NET type awareness for timestamps, paths, environment: supporting capabilities
3	Remoting patterns (Invoke-Command, New-PSSession), authentication, common failure modes
4	Jobs and parallel execution: how the orchestrator runs work across multiple targets simultaneously
5	Module assembly, orchestrator pattern, multi-host coordination: the capstone dress rehearsal
6	Capstone-specific build walkthrough: writing each of the three functions and the orchestrator

> ✅ **The key insight**
> The capstone is not a test of memorization. It is a test of whether you can assemble patterns you have already practiced. By the time you start Friday, you will have written most of the code you need to submit, just against different cmdlets, with different target data. The capstone is the recombination, not new material.

0.5 - Before we start: what you should already know

Why this matters to an operator: This course is titled "Advanced." The title is accurate. If the foundations are not in place on Monday, the gap widens each day until the capstone becomes a rescue operation rather than an assembly task. This section is an honest self-check, so take it seriously.

Prerequisites

The course assumes you can already do the following without looking anything up:

- Open PowerShell, navigate the file system, and execute simple commands
- Use pipelines (`|`) to chain commands
- Read and write variables, including variable expansion inside double-quoted strings
- Use `Get-Help` and `Get-Command` to find cmdlets you do not know
- Write a basic `if` / `else` and a basic `foreach` loop
- Run PowerShell as an administrator when needed, and know why that matters for some cmdlets

If all six of those feel routine, you are in good shape for Monday. Chapter 1 will refresh the patterns that extend from those basics into scripting, and everything from Chapter 2 onward builds on new material.

Self-check

Work through this quick assessment before class on Monday. It takes about ten minutes and gives you an honest read on where your foundations are.

1. **Objects and pipelines.** Can you explain, in one sentence each, the difference between `Get-Process | Select-Object -First 5` and `Get-Process | Format-Table -First 5` ? If you cannot explain why the first is better for scripting, Chapter 1.1 is critical for you.
2. **Discovery.** If someone asks you how to find all cmdlets that deal with scheduled tasks, can you produce the command in under 30 seconds? If not, Chapter 1.2 is worth extra attention.
3. **Variables and types.** What does `'5985' -lt 100` return in PowerShell, and why? If you are not sure, or you are surprised by the answer, Chapter 1.3 will pay off.
4. **Functions.** Can you write a function with a typed, mandatory parameter and a default value, from memory, without looking it up? Chapter 1.6 expects this is refresh-territory for you.
5. **Structured output.** Do you know the difference between returning a hashtable and returning a `[pscustomobject]` ? If that distinction is blurry, Chapter 1.7 is where it gets nailed down.

> 🔧 **If you're uncertain**
> Two resources help: *Learn PowerShell in a Month of Lunches* (4th edition) covers the foundations this course assumes. If you can work through chapters 1-10 of that book in a weekend, you are prepared. Alternatively, spend Sunday evening on the PowerShell documentation's *Getting Started* section on Microsoft Learn. Either will close most gaps.

What is not a prerequisite

You do *not* need to know:

- PowerShell remoting (Chapter 3 teaches this)
- Jobs or parallel execution (Chapter 4 teaches this)
- Modules or orchestration (Chapter 5 teaches this)
- .NET programming or C# (Chapter 2 teaches the .NET exposure you need, no more)
- CIM/WMI deeply (Chapter 1.4 covers what the course uses)

If any of the prerequisite items feel shaky but you are solid on the basics, show up Monday anyway. A little catch-up during Chapter 1 is normal. The course is designed with that in mind.

> ⚠️ **Honest self-assessment is a skill**
> Adults tend to underestimate what they do not know, not overestimate it. If a prerequisite feels 70% (you can do it but not quickly, or you need to look up the syntax), treat it as a gap and shore it up before Monday. "I could probably figure it out" is not the same as "I have this automatic."

0.6 - How to succeed in this course

Why this matters to an operator: Adult learners who have been in the field for years sometimes forget that courses have their own rhythms and failure modes. This section is the stuff that is obvious after you have taken three technical courses, but not obvious beforehand. Five minutes of reading it now saves several hours of frustration later.

During lecture blocks

Morning lecture material is scaffolding. You are not expected to memorize every example or retain every detail. What you *are* expected to do is stay engaged enough to recognize when the afternoon practice asks you to apply something you saw in the morning. The book will still be available for reference; the instructor's demonstrations will not.

Type along with demonstrations when the pace allows. Passive watching and active typing produce very different retention. If you fall behind, the book has the same examples. Catch up during the afternoon block rather than trying to race the instructor.

During practice blocks

The end-of-chapter exercises are the most important part of the day. They are where scaffolding turns into skill. A few principles that separate students who get the most out of the course from students who just get through it:

- **Try it before you ask.** Spend at least five minutes attempting something before flagging the instructor. Often the act of trying surfaces the specific question you actually need answered, which is more useful than a general "I'm stuck."
- **Read your error messages.** PowerShell error messages are unusually good. "The specified computer name is not valid" tells you something different than "Access is denied." Treat those differences as information, not noise.
- **Check the obvious things first.** Is the target up? Are you running the right version of PowerShell? Is your credential actually populated? Most stuck-for-an-hour problems are stuck-for-ten-seconds problems that grew.
- **Do not skip ahead.** If you finish Chapter 2's exercise in 20 minutes and have time left, use the time to redo the exercise differently, or review Chapter 1, or read ahead in the book. Do not start the Chapter 3 exercise early. The chapters are sequenced on purpose.

When something on the range breaks

Range targets go down. This is not a reflection on the range staff. It is a reflection on the fact that the range is complex and many students are using it simultaneously. Two things to do:

1. **Confirm it is actually broken before escalating.** Run `Test-WSMan` against the target. Check if other students are seeing the same issue. A target that is slow is not the same as a target that is down.
2. **Flag it early.** If you do confirm a real problem, tell the instructor immediately. Do not wait until it has blocked you for an hour. Other students are probably hitting the same issue, and the sooner it is reported, the sooner it gets fixed.

When to ask for help

Ask for help when:

- You have spent at least five minutes attempting the problem and can articulate what you have tried
- You have a specific question, not a general "I don't understand"
- An error message is unclear after reading it carefully
- Something on the range seems genuinely broken

Do not ask for help when:

- You have not yet attempted the exercise
- You want the instructor to walk through the whole exercise with you
- You want someone to confirm your answer is right before you try it yourself

The second category is not bad. Those conversations are useful at the end of the day, during wrap-up. They are not a good use of afternoon practice time because they shortcut the learning the exercise is designed to produce.

Between days

This course runs five consecutive days. The pace is deliberate. A few practical notes on making the week work:

- **Do not cram the evening before the capstone.** If Chapters 1 through 5 have gone well, you do not need to cram Thursday night. If they have not, cramming will not save Friday. The capstone is a synthesis exercise, not a memorization exercise. Sleep is a better investment.
- **Review each day's chapter briefly in the evening.** Not a deep re-read. Just the callouts, the check-your-understanding questions, and the exercise you did that afternoon. Fifteen minutes. This is enough to let the material settle.
- **Note questions as they come up.** If something does not click during the day, write it down. Ask the next morning during the first break. Questions accumulate better than they linger.

> ✅ **The mindset that works**
> This course assumes you are a working operator, not a student in the formal sense. That means two things: the instructor respects that you already know how to learn, and you are expected to take responsibility for your own engagement. The material is here. The range is here. The instructor is here. What you do with those five days is on you. Students who show

> up with that posture consistently outperform students who wait for the course to happen to them.

Chapter 1: PowerShell Scripting Foundations Review

> 💡 **Chapter Purpose**
> This chapter refreshes the PowerShell knowledge and habits that make the rest of the course workable. The emphasis is on understanding what commands return, how to inspect and shape that data, how to use discovery tools effectively, and how to turn a working command into a small reusable function.

You already write PowerShell at a working level. That is why you are in this course. Chapter 1 is not a from-scratch introduction. It is a deliberate refresh of the patterns that carry the most weight in later chapters: object inspection, parameter sets, structured output, error handling, and the small habits that separate scripts that survive the field from scripts that only work on the author's machine.

The course environment is built around a student workstation and multiple remote targets. Even when an example begins locally, it should be read with that larger execution model in mind.

System	Role in this course
Ops	Student workstation and execution host
System-1	Remote target
System-2	Remote target
System-3	Remote target
Depot	Remote target and report repository

Scripts will be developed and executed from Ops. As the course progresses, those scripts will gather information from or take action against System-1, System-2, System-3, and Depot. Chapter 1 therefore focuses on the core PowerShell patterns that make later remoting and orchestration reliable.

Learning Objectives

- Explain the difference between display output and the underlying object returned by a command. **(K1)**
- Use help and discovery commands to identify syntax, parameters, examples, properties, and methods. **(K1)**
- Use variables, strings, arrays, hashtables, type inspection, and type casting to support script development. **(K2)**

- Query Windows information with CIM, inspect classes and instances, and retrieve the properties that matter. **(K1)**
- Apply control flow, functions, structured output, and error handling to produce clean script behavior. **(K2)**
- Recognize how local execution context differs from intentionally remote behavior. **(K2)**

Chapter Map

Section	Focus
1.1	Objects, output, and the pipeline
1.2	Help, discovery, and Get-Member
1.3	Variables, strings, arrays, hashtables, types, and casting
1.4	CIM classes, instances, and property retrieval
1.5	Control flow and looping
1.6	Functions, parameters, and execution context
1.7	Returning structured output with PSCustomObject
1.8	Error handling, files, paths, and script troubleshooting
1.9	Chapter review and guided practice
1.10	Exercise

> ✅ **What matters most**
> By the end of this chapter, you should be able to read command output as data, inspect object members, pull useful properties from CIM, write a small function with parameters, and return a clean object that can still be used later in the pipeline. Every one of those skills becomes load-bearing the moment Chapter 3 introduces remoting.

1.1 - Objects, Output, and the Pipeline

Why this matters to an operator: If you treat PowerShell output as text, you end up parsing strings. If you treat it as objects, you get sortable, filterable, exportable data without fighting the shell. Every chapter after this one assumes you are working with objects, not lines.

PowerShell looks like a command-line shell, but it behaves differently from text-only shells in one important way: most commands return **objects**. Those objects carry named properties and methods. The screen shows a formatted view of the object, but the object itself usually contains more information than the display reveals.

That is why PowerShell can filter, sort, export, compare, and transform data without forcing you to parse strings. Once a returned object is understood, a command becomes much easier to script against.

Seeing the difference between display and data

Run a command that returns process objects and look at only a few properties first:

```
Get-Process | Select-Object ProcessName, Id, CPU -First 5
```

The output is shown as rows and columns, but those rows are not plain strings. Each row is a process object with many additional members that are not displayed by default.

Inspecting members with Get-Member

Whenever you need to know what an object contains, send it to `Get-Member`. This is one of the most useful habits in PowerShell.

```
Get-Service | Select-Object -First 1
Get-Service | Select-Object -First 1 | Get-Member
```

The first line shows one returned service. The second line reveals the type, methods, and properties on that object. That tells you what can be selected, compared, exported, or stored in another object later.

> 🔧 **Try it**
> Run both lines above on your local machine. Then compare the output of `Get-Service | Select-Object -First 1` with `Get-Service | Select-Object -First 1 | Get-Member`. The first tells you what the object *looks like*. The second tells you what it *is*. In the field, the second view is the one that unsticks you when a script produces unexpected output.

Properties, methods, and types

A **property** holds data such as a service name or status. A **method** is an action you can call on the object. The **type** identifies what kind of object PowerShell is working with.

```
$Service = Get-Service -Name 'WinRM'
$Service.Status
$Service.DisplayName
$Service.GetType().FullName
```

This pattern is foundational: run the command, save the result to a variable, and inspect the members you care about. Later chapters keep building on that same sequence.

⚠️ A common mistake

Formatting should happen near the end. Once you send objects to `Format-Table` or `Format-List`, you are no longer working with the original objects in a useful way. Filter and select first. Format last.

❓ Check your understanding

What does `Get-Member` tell you that the default table display does not?

1.2 - Help, Discovery, and Get-Member

Why this matters to an operator: You will meet unfamiliar cmdlets mid-task. The ability to discover what a command does, what it expects, and what it returns, without leaving the shell, is what keeps a script-writing session from stalling every time you need to look something up.

PowerShell has a built-in help system that is often the fastest way to work out what a command expects and what it returns. The goal is not to read the entire help page. The goal is to get exactly what you need.

Getting help on a command

```
Get-Help Get-Service -Full
Get-Help Get-Service -Examples
Get-Help Get-Service -Parameter Name
```

`-Full` shows everything. `-Examples` jumps to the examples, which is usually enough for a command you already sort of know. `-Parameter` targets a specific parameter when you need precision.

> 🔧 **Try it**
> Run `Get-Help Get-Service -Examples` on your local machine. Count how many examples are listed. For most core cmdlets, the examples show exactly the patterns you'll use in scripts.

Finding commands you do not know by name

When you know the topic but not the cmdlet, search by verb or noun:

```
Get-Command -Noun Service
Get-Command -Verb Get
Get-Command '*process*'
```

The verb-noun naming convention in PowerShell is not decoration. It makes discovery predictable: if you want to retrieve something, the verb is `Get`. If you want to modify, `Set`. If you want to remove, `Remove`. Knowing that pattern cuts discovery time in half.

Get-Member with filters

A raw `Get-Member` dump can be long. Filter it:

```
Get-Process | Get-Member -MemberType Property
Get-Process | Get-Member -MemberType Method
```

This is how you confirm quickly whether a property you're considering actually exists, or whether the object exposes a method that would do what you're trying to script manually.

⚠️ Update-Help on first use

If `Get-Help` returns only a short header with no examples, the local help files have not been downloaded. Run `Update-Help -Force` from an elevated session once, and help content becomes available offline thereafter.

❓ Check your understanding

If you know you need to retrieve scheduled tasks but do not remember the exact cmdlet, which `Get-Command` parameter helps you find it the fastest?

1.3 - Variables, Strings, Arrays, Hashtables, Types, and Casting

---

Why this matters to an operator: These are not exotic data structures. They are the inputs and outputs of every script you will write. Getting comfortable with their behaviors (especially type coercion and hashtable access patterns) removes a whole category of "it works sometimes" bugs.

Variables hold typed values

A variable in PowerShell holds a value with an underlying type. You do not declare the type explicitly unless you want to. PowerShell infers it from what you assign.

```
$ComputerName = 'System-1'
$Port = 5985
$IsOnline = $true
```

Each of those is a different type: string, integer, boolean. The type matters because operations behave differently. String comparison is not integer comparison.

String interpolation with double quotes

Double-quoted strings expand variables. Single-quoted strings do not.

```
$Message = "Connecting to $ComputerName on port $Port"
$Message
```

This is the most common way to build log lines, file paths, or messages from known values.

Arrays hold ordered collections

```
$Targets = 'System-1', 'System-2', 'System-3'
$Targets[0]                              # first element
$Targets.Count                            # how many items
$Targets += 'Depot'          # append (note: rebuilds the array)
```

⚠ += on arrays is not free

PowerShell arrays are fixed-size. Every `+=` creates a new array and copies all elements. For small lists (a handful of targets), that is fine. For loops building thousands of items, use

`[System.Collections.Generic.List[object]]::new()` instead. This will matter in Chapter 5 when performance shows up on a rubric.

Hashtables hold key-value pairs

```
$SystemInfo = @{
    ComputerName = 'System-1'
    Role         = 'Target'
    Online       = $true
}
$SystemInfo.Role                        # dot notation access
```

Hashtables are the building block for structured output. Chapter 1.7 turns them into " + ic_tag('[pscustomobject]') + ", the thing your capstone functions will return.

Type inspection and casting

When something does not behave as expected, the type is often the culprit:

```
$Port = '5985'                          # string, not int!
[int]$Port                             # cast to integer
$Port.GetType().Name                 # still String
```

Casting with " + ic_tag('[type]') + " converts a value into the specified type. Useful for cleaning up input that came in as string when you needed a number, or vice versa.

> 🔧 **Try it**
> Run `'5985' -eq 5985` and `5985 -eq '5985'` on your local machine. Both return True. Now run `'5985' -lt 100`. It may surprise you. This is why type matters in comparisons: the left operand determines the comparison type, and PowerShell coerces the right operand to match.

> ❓ **Check your understanding**
> What is the practical difference between a hashtable and `[pscustomobject]` made from the same key-value pairs? Why does 1.7 push you toward the second one?

1.4 - CIM Classes, Instances, and Property Retrieval

Why this matters to an operator: CIM is the layer through which PowerShell reads Windows system state: OS details, processes, services, hardware, event logs. When you need to know what a machine *is* rather than what it *returns*, CIM is the entry point. This is the data source for almost every "go check X on all the systems" script you will write in Chapters 4 and 5.

A CIM class is a schema

A CIM class defines what a category of system information looks like. `Win32_OperatingSystem` is a class. A CIM instance is a specific thing of that class, for example, the actual OS on the machine you are querying.

```
Get-CimInstance -ClassName Win32_OperatingSystem
```

This returns one instance because a machine has one operating system. Other classes (like `Win32_Process`) return many instances, one per process.

Selecting the properties you care about

The full OS instance has many properties. Most of them are not useful for a specific task. Select explicitly:

```
Get-CimInstance -ClassName Win32_OperatingSystem |
    Select-Object Caption, Version, LastBootUpTime, OSArchitecture
```

This is the pattern you will use repeatedly: query a CIM class, pipe to `Select-Object`, and take the columns that matter. The capstone's `Get-RemoteProcessInventory` function is essentially this pattern against `Win32_Process`.

> 🔧 **Try it**
> Run `Get-CimInstance -ClassName Win32_OperatingSystem` by itself, then run the Select-Object version. Notice the difference in what gets displayed. Then pipe the first to `Get-Member`. That is how you discover which properties are available to select from.

Discovering classes and their members

```
Get-CimClass -ClassName Win32_Process
Get-CimInstance -ClassName Win32_Process | Get-Member
```

`Get-CimClass` tells you what a class looks like before you query it. Useful when you're trying to find the right class name for what you want to retrieve.

CIM against a remote target

CIM is remoting-aware by default. Adding `-ComputerName` targets another system:

```
Get-CimInstance -ClassName Win32_OperatingSystem -ComputerName 'System-1' -Credential
(Get-Credential)
```

In a domain environment, the current user's Kerberos ticket is used for authentication. Adding `-Credential` lets you target under a different identity when needed.

⚠️ CIM vs WMI

The older cmdlet is `Get-WmiObject`. It still works on Windows PowerShell 5.1 but is not present in PowerShell 7. Use `Get-CimInstance` everywhere. It is the modern, cross-edition equivalent, and it uses a more efficient protocol for remote queries.

✅ Why CIM is important for this course

Every information-gathering function you build in Chapter 5 and the capstone queries a CIM class. `Win32_Process`, `Win32_Service`, `Win32_OperatingSystem`: all CIM. Getting comfortable with the pattern here (query, select, return) pays off every time.

❓ Check your understanding

Why would you use `Get-CimClass` before `Get-CimInstance` if you already know the class name?

1.5 - Control Flow and Looping

Why this matters to an operator: Scripts that touch multiple systems, evaluate state, and branch based on results: that is 90% of what you will write in this course. Control flow is how you express those decisions. The patterns here are small, but they combine into the orchestration work that lands in Chapter 5.

if / else / elseif

```
if ($Service.Status -eq 'Running') {
    "Service is running"
} else {
    "Service is not running"
}
```

Comparison operators in PowerShell use the `-eq`, `-ne`, `-lt`, `-gt`, `-like` style rather than `==` or `!=`. This is because `=` is already reserved for assignment, and it removes ambiguity about whether you meant comparison or assignment.

foreach: iterate a collection

```
foreach ($Target in $Targets) {
    Test-Connection -ComputerName $Target -Count 1 -Quiet
}
```

This is the workhorse loop for "do the same thing to every item in a list", the most common need when you are working against multiple targets.

Where-Object: filter a collection

`Where-Object` filters a collection in the pipeline. Similar idea to a SQL WHERE clause:

```
$Services = Get-Service
$Running = $Services | Where-Object Status -eq 'Running'
$Running.Count
```

Filtering with `Where-Object` is cleaner than looping with `foreach` and manually checking each item. If all you want is "items where X", reach for `Where-Object` first.

> 🔧 **Try it**
> Run `Get-Service | Where-Object Status -eq 'Running' | Measure-Object`. The result tells you how many services are currently running on your machine. Now change the filter to `Status -ne 'Running'`. That pattern, pipeline filter then measure, is the basis for a lot of field triage work.

Assigning loop results to a variable

A powerful pattern: capture the output of a `foreach` loop directly into a variable.

```
$Results = foreach ($Target in $Targets) {
    [pscustomobject]@{
        ComputerName = $Target
        Online       = Test-Connection -ComputerName $Target -Count 1 -Quiet
    }
}
$Results | Format-Table
```

Everything emitted inside the loop becomes part of `$Results`. This is how you build a table of rows, one per target, in a single pass. Chapter 5's parallel orchestrator is this same pattern, just with jobs.

> ⚠️ **foreach vs ForEach-Object**
> The keyword `foreach` is a statement; it iterates a variable. The cmdlet `ForEach-Object` works in the pipeline. They look similar but behave differently. In scripts, prefer the keyword `foreach` for clarity. The pipeline `ForEach-Object` is for one-liners.

> ❓ **Check your understanding**
> You need to run a health check against four targets and build a table of results. Do you reach for `foreach`, `ForEach-Object`, or `Where-Object`, and why?

1.6 - Functions, Parameters, and Execution Context

Why this matters to an operator: A function is the smallest reusable unit of your work. Getting comfortable with parameters, defaults, and mandatory inputs is what moves you from writing one-off scripts to writing tools you can deploy, share, and come back to three months later without having to re-read every line.

A function is just a named script block

```
function Get-SystemStatus {
    Get-CimInstance -ClassName Win32_OperatingSystem
}

Get-SystemStatus
```

That is a valid function. It has no inputs, but it is callable by name and packages a command into something reusable. Not useful in practice, but a valid starting point.

Parameters make functions flexible

```
function Get-SystemStatus {
    param(
        [string]$ComputerName = $env:COMPUTERNAME
    )

    Get-CimInstance -ClassName Win32_OperatingSystem -ComputerName $ComputerName
}

Get-SystemStatus                          # uses default (local machine)
Get-SystemStatus -ComputerName 'System-1'
```

Two things happened here:

- The `param` block declares `$ComputerName` as an input, typed as `[string]`, with a default value.
- Because it has a default, calling the function without any argument still works.

Typed parameters catch a whole class of bug early. If someone passes an integer to a `[string]` parameter, PowerShell will either cast cleanly or complain loudly. Both are better outcomes than silent type confusion.

> 🔧 **Try it**
> Define `Get-SystemStatus` as shown above on your local machine. Call it with no arguments first, then call it with `-ComputerName $env:COMPUTERNAME` explicitly. Confirm both return the same result. Then try passing a bad value, something like `-ComputerName 42`. Watch how PowerShell handles the type mismatch.

Mandatory parameters and multiple inputs

```
function Get-SystemStatus {
    param(
        [Parameter(Mandatory)]
        [string]$ComputerName,

        [pscredential]$Credential
    )

    if ($Credential) {
        Get-CimInstance -ClassName Win32_OperatingSystem -ComputerName $ComputerName -
Credential $Credential
    } else {
        Get-CimInstance -ClassName Win32_OperatingSystem -ComputerName $ComputerName
    }
}
```

`[Parameter(Mandatory)]` forces the caller to supply a value. If omitted, PowerShell prompts interactively rather than silently using null.

Execution context: where the function is running

A function runs wherever the PowerShell session is running. That sounds obvious, but becomes important in Chapter 3 when you send a function body into a remote session. The function does not follow the variables from the caller automatically. That constraint shapes how you design parameters.

> ✅ **The capstone connection**
> Every function you write for the capstone follows the same skeleton as the examples above: `param` block with `[string]$ComputerName` and `[pscredential]$Credential`, a try/catch wrapping a `Get-CimInstance` or `Invoke-Command` call, and a structured return. Get this skeleton muscle-memorized now. You will write it many times before July.

> ⚠️ **Functions are not scripts**
> A `.ps1` file is a script. A `function` inside it is a named block you can call. They are not the same. Scripts execute top to bottom when invoked. Functions execute only when called by

name. Your capstone module holds functions; your orchestrator is a script that imports the module and calls those functions.

? Check your understanding

What happens if you mark a parameter `[Parameter(Mandatory)]` but also give it a default value? Is that a useful combination, or is one of the two ignored?

1.7 - Returning Structured Output with PSCustomObject

Why this matters to an operator: Unstructured output cannot be parsed, sorted, exported, or passed to the next function. Structured output can. The difference between a script that is useful once and a tool that gets reused across missions is almost always whether the output has a shape.

Why not just use Write-Host?

```
Write-Host "ComputerName: System-1"
Write-Host "Status: Online"
Write-Host "OS: Windows 11"
```

That puts text on the screen. It does not produce data. If another script needs to know whether System-1 is online, there is nothing to check, just characters that already went to the console. This is also why `Write-Host` is often called "not a command, a sin." It displays without returning.

PSCustomObject returns data with shape

```
[pscustomobject]@{
    ComputerName = 'System-1'
    Status       = 'Online'
    OS           = 'Windows 11'
}
```

Now the result is an object with three named properties. It can be assigned to a variable, filtered, sorted, exported to CSV or JSON, or fed to another function. It behaves like every other object in PowerShell.

Building a table of objects in a loop

```
$Results = foreach ($Target in $Targets) {
    [pscustomobject]@{
        ComputerName = $Target
        Online       = Test-Connection -ComputerName $Target -Count 1 -Quiet
        TimeStamp    = Get-Date
    }
}
$Results | Export-Csv -Path '.\results.csv' -NoTypeInformation
```

This is the core pattern for any "check X on every target" script. Build an object per target, assign the whole collection to a variable, and then do whatever with it: export, filter, format.

> 🔧 **Try it**
> Build a small version of the loop above against three fake targets (you can just use strings like `'System-1','System-2','System-3'`). Assign the result to `$Results`. Then try `$Results | Where-Object Online -eq $true` and `$Results | Sort-Object ComputerName`. Every one of those operations works because the output has shape.

The capstone wrapper pattern

Your capstone functions return a single `[pscustomobject]` with a specific shape:

```
[pscustomobject]@{
    ComputerName = $ComputerName
    Function     = 'Get-RemoteProcessInventory'
    Status       = 'Success'
    Data         = $ProcessArray     # the actual data lives here
    Error        = $null
}
```

The wrapper carries status metadata (ComputerName, Status, Error) and holds the actual data in a `.Data` property. This shape is intentional: it lets the orchestrator know whether a function succeeded before it tries to do something with the data.

> ⚠️ **Watch the object shape when counting**
> The wrapper above is a scalar. `$Result.Count` returns 1 even if `$Result.Data` contains 500 process objects. To count the actual data, use `$Result.Data.Count`. This trap appears again in Chapter 5.

> ✅ **What matters here**
> If you take one habit from Chapter 1, take this one: every function returns an object, and every object has named properties. Never return ad-hoc text. Never return different shapes from the same function depending on the path. Consistency in return shape is what makes the rest of your code easy to write.

> ❓ **Check your understanding**
> You have two functions that both return information about a system, but they return objects with different property names. What problem will this cause when the orchestrator tries to combine results?

1.8 - Error Handling, Files, Paths, and Script Troubleshooting

Why this matters to an operator: Scripts fail. On real ranges, failure is constant: unreachable hosts, missing permissions, stale DNS, services in the wrong state. The question is not whether your script will hit an error. The question is whether the error produces a useful result or a silent zero. Error handling is how you answer that question.

Scripts become more reliable when they can recognize failure clearly. PowerShell does not treat every error the same way. Some errors are non-terminating, which means a command reports the issue but keeps going. Others are terminating, which means execution stops unless the error is handled.

try and catch, with -ErrorAction Stop

```
try {
    Get-CimInstance -ClassName Win32_OperatingSystem -ComputerName 'DoesNotExist' -
ErrorAction Stop
}
catch {
    $_.Exception.Message
}
```

The important detail here is `-ErrorAction Stop`. Without it, many cmdlets write a non-terminating error and keep going, which means your `catch` block never fires. The fix is explicit: tell the cmdlet to treat its errors as terminating so you can actually catch them.

The success/failure structured-output pattern

Combining structured output (1.7) with error handling (here) produces the pattern every capstone function uses:

```
try {
    $Data = Get-CimInstance -ClassName Win32_OperatingSystem -ComputerName $ComputerName -
ErrorAction Stop
    [pscustomobject]@{
        ComputerName = $ComputerName
        Status       = 'Success'
        Data         = $Data
        Error        = $null
    }
}
catch {
    [pscustomobject]@{
        ComputerName = $ComputerName
        Status       = 'Failed'
        Data         = $null
        Error        = $_.Exception.Message
    }
}
```

Regardless of whether the query succeeded or failed, the function returns an object with the same shape. The orchestrator does not need a separate branch for failures. It just checks the `Status` property and routes accordingly.

> ✅ **Why this pattern matters**
> An orchestrator that runs this function against 50 hosts gets back 50 objects: some with `Status = 'Success'`, some with `Status = 'Failed'`. Instead of 50 hostnames to investigate, the operator has a one-column filter that tells them exactly which 3 to look at. This is what "scales" actually means in PowerShell scripting.

Files and paths

Path handling matters because scripts often create logs, reports, or temporary files. Building paths explicitly is cleaner than relying on assumptions about the current working directory.

```
$ReportPath = Join-Path -Path $PSScriptRoot -ChildPath 'snapshot.csv'
if (Test-Path $ReportPath) {
    Remove-Item $ReportPath -Force
}
```

`$PSScriptRoot` is especially useful in scripts because it refers to the folder where the script itself is stored. That makes file behavior predictable across machines, even when the user is running from a different current directory.

> 🔧 **Try it**
> Save a short script (2–3 lines is fine) that prints `$PSScriptRoot`. Run it from the same

> directory. Then run it from a different directory (use `cd` elsewhere, then invoke the script by full path). The value should remain the same: the folder the script lives in, not the folder you're currently in.

A short troubleshooting habit

When a script misbehaves, check four things in order before you start rewriting:

1. The parameter values actually passed in (not what you intended to pass)
2. The type of the object you're operating on
3. The execution context (local vs remote, elevated vs not)
4. The exact error message (not your interpretation of it)

Most "weird" failures resolve at one of those four checks. The rewrite-first instinct usually produces new bugs before fixing the original.

⚠️ A useful troubleshooting habit

When something fails, check the parameter values, the object type, the current execution context, and the exact error message before rewriting the function.

❓ Check your understanding

Why does adding `-ErrorAction Stop` change what `try/catch` can do? What happens without it?

1.9 - Chapter Review and Guided Practice

Why this matters to an operator: This section is the sanity check before the graded exercise in 1.10. If any of the patterns below feel unfamiliar, now is the time to go back and work through the section that introduced them. The rest of the course assumes these are automatic.

The patterns to have internalized

Pattern	Where it was introduced
Pipe to `Get-Member` to inspect objects	1.1, 1.2
Use `Get-Help -Examples` for quick reference	1.2
Declare typed parameters with defaults	1.6
Return `[pscustomobject]` with named properties	1.7
Wrap queries in `try/catch` with `-ErrorAction Stop`	1.8
Use `$PSScriptRoot` for script-relative paths	1.8
Filter with `Where-Object` before formatting	1.5

A complete example that uses all of it

Here is a small function that combines every pattern from this chapter. Read it as a single coherent example, not a collection of techniques:

```powershell
function Get-OSInfo {
    param(
        [string]$ComputerName = $env:COMPUTERNAME
    )

    try {
        $OS = Get-CimInstance -ClassName Win32_OperatingSystem -ComputerName $ComputerName
-ErrorAction Stop

        [pscustomobject]@{
            ComputerName = $ComputerName
            Caption      = $OS.Caption
            LastBoot     = $OS.LastBootUpTime
            Status       = 'Success'
            Error        = $null
        }
    }
    catch {
        [pscustomobject]@{
            ComputerName = $ComputerName
            Caption      = $null
            LastBoot     = $null
            Status       = 'Failed'
            Error        = $_.Exception.Message
        }
    }
}
```

Every line here was discussed in an earlier section. If any part of it feels unclear, revisit the section listed in the table above.

> 🔧 **Try it: guided practice**
> Paste the function above into your local PowerShell session. Call it three ways:
> 1. `Get-OSInfo` (no arguments, uses default)
> 2. `Get-OSInfo -ComputerName 'localhost'`
> 3. `Get-OSInfo -ComputerName 'DoesNotExist'`
>
> All three should return objects with the same shape. Only the third should have `Status = 'Failed'`. Then pipe all three results to `Format-Table` and confirm that the columns line up cleanly.

Self-assessment: you are ready for 1.10 when...

- You can write a typed " + ic_tag("param") + " block from memory, including " + ic_tag("[Parameter(Mandatory)]") + " and default values
- You can recall the difference between a hashtable and a " + ic_tag("[pscustomobject]") + " and when you would use each

- You can explain what " + ic_tag("-ErrorAction Stop") + " does and why " + ic_tag("catch") + " needs it for many cmdlets
- You know how to inspect what a command returns before trying to filter or select from it
- You know how to use " + ic_tag("$PSScriptRoot") + " to make scripts location-independent

If any of those feel shaky, go back. The exercise in 1.10 combines all of them, and the rest of the course assumes they are automatic.

> ✅ **The capstone bridge**
> The function above is a simpler version of what your capstone functions will look like. Same param block. Same try/catch. Same wrapper object. The only things that change are the CIM class, the properties selected, and the caller (local in 1.10, remote in Chapter 5+). The skeleton is stable, and that is intentional. Practice it until it is automatic.

1.10 - Exercise

🔧 Chapter 1 Exercise: Build a System Info Reporter

Put together the skills from this chapter into one short script. Work from Ops.

1. Write a function called `Get-SystemSnapshot` that accepts a `-ComputerName` parameter with a default value of `$env:COMPUTERNAME`.
2. Inside the function, use `Get-CimInstance Win32_OperatingSystem` to retrieve the OS caption and last boot time.
3. Return a `[pscustomobject]` with properties: **ComputerName**, **OSCaption**, **LastBoot**, **Status**, and **Error**.
4. Wrap the CIM query in a `try/catch` block. On failure, return a failure object with the exception message.
5. Call your function twice (once with no argument for local, once with an intentionally bad computer name) and pipe both results to `Format-Table`.

Success criteria: Both the success and failure rows display cleanly. The error message is captured in the object, not thrown to the console.

> ✅ **Why this exercise**
> This is the minimum viable version of a capstone function. It exercises typed parameters, CIM queries, try/catch with -ErrorAction Stop, structured return objects, and consistent output shape across success and failure paths. If you can build this from memory, you have the foundation. The only differences between this and your capstone deliverables are the specific CIM classes and the fact that those are called remotely.

> ⚠️ **Before you submit**
> Run your function with `Get-Member` to confirm the return type is `System.Management.Automation.PSCustomObject` and that both the success and failure paths return objects with identical property names. Mismatched shapes between success and failure paths are the most common rubric-failing mistake on this exercise.

Chapter 2: .NET for PowerShell Practitioners

> 💡 **Chapter Purpose**
> This chapter makes the PowerShell/.NET relationship explicit. The goal is not to turn you into a .NET developer. The goal is to help you recognize the object and type patterns that make PowerShell more powerful, and to know when reaching for a .NET type gives you something PowerShell's cmdlets do not.

You already know from Chapter 1 that PowerShell returns objects, not plain text. This chapter names what those objects actually are: they are .NET objects. Every process, service, date, or file-info result you work with is an instance of a .NET class with a type name, properties, and methods.

This matters because recognizing the .NET layer underneath lets you find capabilities that are not exposed as standalone cmdlets (timestamps, path construction, machine info, math operations), and it lets you read scripts (including adversary scripts) more accurately.

Learning Objectives

- Explain how .NET types appear in PowerShell output and commands. **(K1)**
- Inspect object properties and methods by using discovery tools. **(K1)**
- Create structured output by using `[pscustomobject]`. **(K1)**
- Distinguish between static member syntax and instance method usage. **(K1)**
- Apply practical .NET patterns commonly used in PowerShell scripts. **(K1)**

Chapter Map

Section	Focus
2.1	PowerShell and .NET
2.2	Properties, methods, and type names
2.3	Static vs instance members
2.4	Useful .NET types in scripts
2.5	Assemblies and namespaces
2.6	Practical examples
2.7	Chapter exercise

> ✅ **What matters most**
> By the end of this chapter, you should be able to read a script that uses .NET types (square brackets, double colons) and know what it's doing. You should also be able to reach for a .NET type when a cmdlet does not exist for what you need. Beyond those two abilities, stay out of .NET unless a problem actively demands it.

2.1 - PowerShell and .NET

Why this matters to an operator: When a script works on your Ops workstation but fails on a different host, the .NET edition under PowerShell is one of the first things to check. Knowing how to confirm it, and understanding why the answer changes between PowerShell 5.1 and PowerShell 7, eliminates a whole category of "why does this break over there" debugging.

PowerShell is built on top of the .NET runtime. This is not trivia. It is the reason PowerShell commands can return rich objects instead of plain text. Every object you work with in PowerShell is a .NET object with a type name, properties, and methods.

Checking the PowerShell and .NET context

Run this to see what version and edition you are on:

```
$PSVersionTable
```

Two lines in the output matter most. The `PSVersion` line shows the PowerShell version. The `PSEdition` line shows whether you are running Windows PowerShell (`Desktop` , built on .NET Framework) or PowerShell 7+ (`Core` , built on modern .NET). These two editions behave differently enough that the distinction matters when a script works on one host and fails on another.

> ⚠️ **CLRVersion behavior differs by edition**
>
> In Windows PowerShell 5.1, a `CLRVersion` property also appears in `$PSVersionTable` and reports the .NET Framework version. That property is not present in PowerShell 7. If you need the exact runtime version on PowerShell 7, use the command below instead.

```
[System.Runtime.InteropServices.RuntimeInformation]::FrameworkDescription
```

> 🔧 **Try it**
>
> Run `$PSVersionTable` on your Ops workstation. Confirm `PSEdition` shows `Core` and `PSVersion` shows 7.x. Then run the `FrameworkDescription` command. Note what it returns. If you ever need to prove which .NET is active on a host, that single line is your evidence.

Why this connects to everything else

Every object you see in PowerShell output (processes, services, dates, file info) is backed by a .NET class. The `PSEdition` you are running determines which .NET classes are actually available, which is one reason a script can succeed on one host and fail on another.

✅ Why this matters

When you pipe `Get-Date` to `Get-Member`, you are looking at the .NET type `System.DateTime` and its members. Understanding this connection helps you find capabilities that are not exposed as standalone cmdlets. Understanding which *edition* of .NET you are on helps you avoid writing scripts that work on your workstation but fail on a PS 5.1 host, or vice versa.

❓ Check your understanding

If a teammate reports a script failing on a Windows Server 2016 host with a Windows PowerShell 5.1 default, what's the first question you'd ask them before looking at the script itself?

2.2 - Properties, Methods, and Type Names

Why this matters to an operator: Every troubleshooting session that goes beyond "why didn't my cmdlet work" involves asking the object itself what it can do. Properties tell you what data the object carries. Methods tell you what actions it can perform. Type names tell you what class the object is. Knowing how to pull those three pieces of information is the foundation of everything in Chapter 2.

Discovering a type name

```
$Date = Get-Date
$Date.GetType().FullName
```

That returns `System.DateTime`, the .NET class behind every date object PowerShell hands you. Once you know the type name, you can search Microsoft's .NET documentation for every method and property that class supports, even ones not obvious from `Get-Member` output.

Listing properties and methods

```
$Date | Get-Member -MemberType Property
$Date | Get-Member -MemberType Method
```

Filtering `Get-Member` by `-MemberType` is the fastest way to cut through the noise. You almost never need to see all members at once. You usually need to answer one of two questions: "what data does this object have" (property) or "what can I do with this object" (method).

> 🔧 **Try it**
> Run the property and method queries against a process object: `Get-Process pwsh | Get-Member -MemberType Method`. Count how many methods are available. Most PowerShell users never realize process objects expose things like `Kill()` and `WaitForExit()`. That discovery is the point of Get-Member.

Accessing properties vs calling methods

Properties use dot notation with no parentheses. Methods use dot notation with parentheses.

```
$Date.DayOfWeek                          # property -- no parentheses
$Date.AddDays(7)                          # method -- parentheses required
```

⚠️ Common mistake: calling a method without parentheses

If you write `$Date.AddDays` without parentheses, PowerShell returns a description of the method instead of calling it. You get back an object describing the method's signature, which looks like output but is not what you wanted. Always include parentheses when calling a method, even if there are no arguments: `$Date.GetType()`, not `$Date.GetType`.

❓ Check your understanding

When you inspect an object with `Get-Member` and see a member labeled Property vs one labeled Method, how do you know which syntax to use? And what does PowerShell do if you guess wrong?

2.3 - Static vs Instance Members

Why this matters to an operator: Half the .NET usage you will read in real PowerShell scripts is static member calls, the square-bracket-double-colon syntax. Knowing what that syntax *is* lets you read those scripts without guessing. It also lets you find quick info like the current user, the machine name, or the current timestamp without wrapping each call in a cmdlet.

Instance members: need an object

An instance member is called on a specific object. You need an object first.

```
$Date = Get-Date                    # create an instance first
$Date.AddDays(1)                     # call method on that instance
$Date.DayOfWeek                      # access property on that instance
```

Static members: called on the type with ::

A static member belongs to the type itself. No object needed:

```
[datetime]::Now                      # no object needed
[environment]::MachineName          # no object needed
[environment]::UserName             # no object needed
```

The double-colon `::` is the static member operator. When you see it in a script, you know the member belongs to the type, not to an instance.

> 🔧 **Try it**
> Run all three of the static examples on your Ops workstation. Notice that no variable is declared, no instance is created. PowerShell just evaluates the expression and returns a value. This is why static members are so common in scripts: they get you a value in one line without ceremony.

> ✅ **Recognizing static vs instance in real scripts**
> When you read a script and see `[something]::something`, you're looking at a static call. When you see `$variable.something`, you're looking at an instance call. These are the only two shapes you need to recognize in Chapter 2. Everything else is application.

> ❓ **Check your understanding**

Which of the examples in this section require an existing object in a variable, and which do not? More specifically: why does `[datetime]::Now` work without any setup but `$Date.AddDays(1)` requires `$Date` to exist first?

2.4 - Useful .NET Types in Scripts

Why this matters to an operator: Five .NET types show up repeatedly in real PowerShell scripting. Knowing these by name, and the one or two things each one is useful for, saves you from looking for a cmdlet that does not exist. You are not memorizing the full .NET class library. You are memorizing a small, high-leverage set.

Type	What it gives you	Example
`[datetime]`	Timestamps, date math	`[datetime]::Now`
`[environment]`	Machine name, user, OS info	`[environment]::MachineName`
`[System.IO.Path]`	Path manipulation	`[System.IO.Path]::Combine('C:\Tools', 'Sysmon')`
`[math]`	Rounding, max/min	`[math]::Round(3.14, 2)`
`[System.Net.Dns]`	DNS lookups	`[System.Net.Dns]::GetHostName()`

Putting a few to work

```
# Build a timestamp for logging
[datetime]::Now.ToString('yyyy-MM-dd HH:mm:ss')

# Safe path joining
[System.IO.Path]::Combine('C:\Tools', 'Sysmon', 'Sysmon64.exe')

# Get the current machine name
[environment]::MachineName
```

Read each block as "I need X; the cmdlet for X is not obvious; here is the .NET one-liner." That is the pattern. You are not reaching into .NET because it is cool. You are reaching into it when the cmdlet does not exist or is more awkward than the .NET equivalent.

> 🔧 **Try it**
> Run `[System.IO.Path]::Combine('C:\Tools', 'Sysmon', 'config.xml')` on your Ops workstation. Notice the result handles the backslashes correctly. You did not have to build the string by hand. Now try it with a forward slash: `[System.IO.Path]::Combine('/tmp',`

> `'report.csv')` . On Windows, this also works because .NET normalizes the separators. This is why `[System.IO.Path]::Combine()` is safer than string concatenation for paths.

> 💡 **Namespace shorthand**
> PowerShell accepts namespace shorthand for common types: `[IO.Path]` resolves to `[System.IO.Path]` , `[Net.Dns]` resolves to `[System.Net.Dns]` . Both forms work. This course uses the full name in reference material and the shorthand when space is tight in a script. Scripts you read in the wild will use both. Know that they are equivalent.

> ❓ **Check your understanding**
> If your script needs to build a log filename that includes a timestamp and then write it to `C:\Logs` , which two of the five types in the table above would you use, and for which part?

2.5 - Assemblies and Namespaces

Why this matters to an operator: Assemblies and namespaces are .NET plumbing. For most day-to-day scripting, you will not think about them. For the occasional case where you need to load something that is not available by default, knowing what's happening beneath the surface makes the fix obvious instead of mysterious.

Assemblies package compiled .NET code. Namespaces organize types into groups. In most day-to-day scripting, you will use built-in types without needing to think about either. Types like `[datetime]`, `[string]`, and `[math]` are always available because their assemblies are loaded by default.

When you do need to load something

When a type is not available, loading its assembly brings it in:

```
try {
    Add-Type -AssemblyName System.Windows.Forms -ErrorAction Stop
    'Assembly loaded successfully'
}
catch {
    $_.Exception.Message
}
```

The `try/catch` wrapper tells you cleanly whether the load worked. If the assembly does not exist on this host (or on this edition of PowerShell), the catch fires and you know to stop rather than failing with an obscure error later.

> ✅ **Course scope**
> Assemblies and namespaces are background knowledge for this course. If you find yourself deep in .NET assembly loading during the capstone, you have probably overcomplicated the solution. The built-in types cover everything the capstone requires.

> ❓ **Check your understanding**
> If you run `Add-Type -AssemblyName System.Windows.Forms` on PowerShell 7 on Windows, it works. Run the same command on PowerShell 7 on Linux and it fails. Why? What's the single-word answer?

2.6 - Practical Examples

Why this matters to an operator: The hard part of using .NET in PowerShell is not using it. The hard part is knowing when *not* to. This section shows both sides: where .NET types make a script cleaner, and where reaching for .NET makes a simple task harder to read.

Combining types into a structured result

Each .NET static call on its own is one line of information. Combined inside `[pscustomobject]`, they become a single reusable result:

```
[pscustomobject]@{
    TimeStamp   = [datetime]::Now
    Workstation = [environment]::MachineName
    UserName    = [environment]::UserName
    SysmonPath  = [System.IO.Path]::Combine('C:\Tools', 'Sysmon', 'Sysmon64.exe')
}
```

That pattern, static calls as inputs to a structured object, is close to what every capstone function will do. The function just substitutes a real CIM query in place of the static calls.

> 🔧 **Try it**
> Paste the object above into your Ops workstation. Assign it to `$Snapshot` and inspect: `$Snapshot.SysmonPath`, `$Snapshot | Format-List`. Confirm the properties carry the values the static calls produced. This is what "structured output" means in practice.

When NOT to use .NET directly

If a cmdlet already does the job, prefer the cmdlet. It is more readable and more discoverable.

```
# Prefer this:
Test-Path 'C:\Tools\Sysmon\Sysmon64.exe'

# Over this (same result, harder to read):
[System.IO.File]::Exists('C:\Tools\Sysmon\Sysmon64.exe')
```

Both lines return the same result. The first is clearly PowerShell. The second requires you to remember a .NET namespace, a type name, and a method name, for no benefit.

> ⚠️ **The readability test**

When you consider using a .NET type, ask: if I hand this script to a teammate, will they recognize what it does in under five seconds? If the cmdlet version is faster to recognize, use the cmdlet. If the .NET version is actually clearer (path combining, for example), use .NET. The goal is clarity, not cleverness.

✅ **The capstone connection**

The structured object pattern above is the shape of every Get-* function you will write in the capstone. Timestamp, machine name, status fields, and data payload, all assembled inside a single `[pscustomobject]` and returned. The static call syntax is a small detail. The composition pattern is the lesson.

2.7 - Chapter 2 Exercise

Why this matters to an operator: This exercise exists to confirm one thing: that you can reach for a .NET type when a cmdlet is not obvious, and still return a clean structured object. Every pattern is a building block for your capstone functions.

> 💡 **Scenario**
> You need to create a script that audits the current execution environment. It must identify the underlying PowerShell version, calculate a "Review Date" 30 days into the future, and identify the current user and machine.

Exercise Requirements

Using the .NET discovery and application skills from this chapter, complete the following tasks:

- **Discovery:** Identify the full .NET type name of the `$PSVersionTable` object.
- **Static Members:** Use `[datetime]` to get the current time and `[environment]` to get the current username.
- **Instance Methods:** Take the current date object and use a method to calculate a date exactly 30 days in the future.
- **Math:** Use the `[math]` type to round the number `124.76` to the nearest whole integer.
- **Output:** Return all these findings in a single `[pscustomobject]`.

Exercise Skeleton: Test-Environment.ps1

```
# 1. Discover the type of $PSVersionTable
$TableType = #

# 2. Get current system info using static members (::)
$Now  = #
$User = #

# 3. Calculate a review date using an instance method (.)
$ReviewDate = #

# 4. Use [math] to round 124.76
$RoundedValue = #

# 5. Return the results as a structured object
[pscustomobject]@{
    NET_Type    = $TableType
    CurrentUser = $User
    ReviewDue   = $ReviewDate
    RoundedNum  = $RoundedValue
}
```

✅ **Self-check**

Before you consider this exercise complete, confirm three things: (1) every variable holds a real value, not `$null`; (2) the final `[pscustomobject]` returns one clean row when piped to `Format-Table`; (3) the `ReviewDue` field shows a real date 30 days from now, not today's date with a string appended.

⚠️ **Common gotcha**

Task 3 asks you to use an instance method. A common mistake is calling `[datetime]::Now.AddDays(30)`, which works, but the spirit of the exercise is to capture the current date into a variable first and then call the instance method on the variable. The static-then-instance pattern is the one the capstone needs you to recognize.

Chapter 3: PowerShell Remoting and Operational Context

> 💡 **Chapter Purpose**
> This chapter takes you from local PowerShell to remote PowerShell. Almost every operationally interesting script runs across multiple systems. The patterns here, including WinRM, sessions, authentication, scope, and credentials, are what make that possible. They are also the source of the most common "works on my machine, fails on yours" failures, which is why the chapter spends real time on diagnosis.

Through Chapter 2, every example ran on your Ops workstation. That was deliberate. The foundations needed to be solid before adding the network. Chapter 3 introduces the network. By the end of the chapter, you will be able to run PowerShell commands on System-1, System-2, System-3, and Depot from your Ops workstation, handle the authentication that makes that possible, and recognize the failure modes that consistently trip up new operators.

This chapter is also where security context becomes part of the conversation. Remote execution is exactly what defenders watch for, and what adversaries abuse. Section 3.6 names the defensive primitives every PowerShell operator should know exist, even if you are not the one configuring them.

Learning Objectives

- Configure and use WinRM-based remoting in a domain environment. **(K2)**
- Open, use, and close persistent remote sessions. **(K2)**
- Pass local values into remote script blocks correctly. **(K2)**
- Recognize and work around the double-hop problem. **(K1)**
- Diagnose the most common remote execution failures from their symptoms. **(K2)**
- Identify language mode behavior and its operational implications. **(K1)**
- Describe how PowerShell is abused and how defenders detect that abuse. **(K1)**

Chapter Map

Section	Focus
3.1	Debugging mindset
3.2	WinRM and remoting basics
3.3	Authentication, delegation, and the double-hop problem
3.4	Common remote execution issues
3.5	Language modes

Section	Focus
3.6	PowerShell abuse and defensive context
3.7	DSC, APIs, and platform integration
3.8	Chapter exercise: Remote Connectivity Validator

> ✅ **What matters most**
> Three skills carry the most weight from this chapter into the capstone: opening a session with `New-PSSession` and a credential, passing variables in with `$using:`, and reading an "Access denied" error correctly. If you internalize those three patterns, every remote script you write afterward starts from a solid base.

3.1 - Debugging Mindset

Why this matters to an operator: The hardest scripting failures are not the loud ones. They are the silent wrong answers, the inconsistent results, the "works on my machine" problems. A debugging mindset is not a tool. It is a habit of asking specific questions before reaching for the keyboard.

Before any specific tool or technique, debugging starts with a discipline: when something does not work, you systematically narrow the failure. You do not start by rewriting. You start by checking what you can verify.

The four checks before you rewrite anything

When a script misbehaves, check these four things in order, every time:

1. **The parameter values actually passed in.** Not what you intended to pass. Not what looks right. The actual values as PowerShell sees them.
2. **The type of the object you are operating on.** A `[string]` that looks like a number is still a string. A scalar `[pscustomobject]` is not an array, even if it "feels" like one.
3. **The execution context.** Local or remote? Elevated or not? Which user? Which PowerShell edition?
4. **The exact error message.** Not your interpretation of it. The literal text. PowerShell error messages are unusually good, so read them.

Most "weird" failures resolve at one of those four checks. The instinct to rewrite first usually produces new bugs before it fixes the original.

Tools for those four checks

```
$Service = Get-Service -Name 'WinRM'
$Service | Get-Member
$Service.Status
$Service.GetType().FullName
```

The pattern from Chapter 1 holds: capture into a variable, inspect with `Get-Member`, examine specific properties. This is not just for learning. It is your debugging tool when something returns the wrong shape.

```
whoami                                  # who am I authenticated as?
$PID                                    # what process is this?
[environment]::Is64BitProcess          # what bitness?
$PSVersionTable.PSEdition             # Desktop or Core?
```

These four lines tell you the execution context in seconds. Run them first when a script behaves unexpectedly on a different host.

> 🔧 **Try it**
> Run all four context checks above on your Ops workstation. Note what they return. Then imagine you logged into a server and ran the same four lines. Which of the four answers would tell you the most about whether your script will behave the same way? (Hint: not all four are equally informative.)

> ⚠️ **The most expensive habit**
> Rewriting a script before understanding why it failed creates a new script with new bugs and the original problem still hidden somewhere. If you find yourself reaching for the editor before completing the four checks, stop. Read the error. Check the values. Inspect the type. Then decide whether a rewrite is actually warranted.

> ✅ **Why this leads the chapter**
> Remote execution multiplies failure modes. Auth, network, scope, credentials, language mode: any of them can fail silently or loudly. The four-check habit applies everywhere in this chapter. When something does not work, do not guess. Check.

> ❓ **Check your understanding**
> A teammate reports their script "works locally but doesn't work remotely." Which of the four checks is most likely to identify the real problem first?

3.2 - WinRM and Remoting Basics

Why this matters to an operator: WinRM is the protocol that makes everything else in this chapter possible. The cmdlets are simple. The conceptual model (local vs remote scope, sessions vs ad-hoc commands, how variables cross the boundary) is where mistakes happen. Get the model right and the cmdlets fall into place.

PowerShell remoting uses Windows Remote Management (WinRM) to run commands on another system. WinRM is a service that listens on a network port, authenticates incoming requests, and runs PowerShell on the target. From your Ops workstation, you send PowerShell to a remote system, and the result comes back as objects, not text.

Confirming a target is reachable

```
Test-WSMan 'System-1'
```

`Test-WSMan` returns a small response object if the target's WinRM service is up and reachable. If you get an error, the target is either down, blocked at the firewall, or not configured for WinRM. This is the first command to run when remoting fails, before anything else.

Ad-hoc remote command with Invoke-Command

```
Invoke-Command -ComputerName 'System-1' -ScriptBlock {
    Get-Service -Name 'WinRM'
}
```

This sends the script block to System-1, runs it there, and brings the result back to your local session. One-shot operations like checking a service or pulling a quick value work well this way.

> 🔧 **Try it**
> Confirm your Ops can reach all four targets: `'System-1','System-2','System-3','Depot' | ForEach-Object { Test-WSMan $_ }`. Any target that throws is one that will block your Chapter 5 exercises. Better to know now than Wednesday.

Persistent session with New-PSSession

For multiple operations against the same target, opening a persistent session is more efficient than repeated ad-hoc commands. The session stays open, multiple `Invoke-Command` calls reuse it, and you close it when you are done.

```
$Session = New-PSSession -ComputerName 'System-1'
Invoke-Command -Session $Session -ScriptBlock { Get-Process -Name 'pwsh' }
Invoke-Command -Session $Session -ScriptBlock { Get-Service -Name 'WinRM' }
Remove-PSSession $Session
```

Sessions also enable patterns that ad-hoc `Invoke-Command` cannot handle, like copying files between sessions, entering interactive sessions, and chaining state-dependent operations. Chapter 5 builds heavily on persistent sessions.

> ⚠️ **Always close sessions**
>
> Open sessions consume resources on both Ops and the target. If you test a script ten times without cleanup, you have ten orphaned sessions. `Get-PSSession | Remove-PSSession` is a good habit before walking away from your workstation. Chapter 5.9 covers cleanup in detail.

Passing local values into remote script blocks: $using:

A common mistake: writing a remote script block that references a local variable, and being surprised when the variable is empty inside the remote session. The remote session does not see your local scope. Variables do not cross the boundary automatically.

The fix is the `$using:` scope modifier. When a remote command needs a local value, use `$using:` to copy that value into the remote scope:

```
$ServiceName = 'Spooler'                        # defined locally on Ops
Invoke-Command -ComputerName 'System-1' -ScriptBlock {
    Get-Service -Name $using:ServiceName        # pull the local value into the remote
session
}
```

Without `$using:ServiceName`, the remote session looks for a variable named `$ServiceName` on the remote machine, finds nothing, and `Get-Service` fails or returns nothing.

> 💡 **When to use $using:**
>
> Use `$using:` when the value already exists locally and the remote command only needs to read it. `$using:` sends a local value into the remote script block. It does not create a permanent variable on the remote machine. Once the script block finishes, the value is gone.

> ✅ **The capstone connection**
>
> Every capstone function uses `Invoke-Command` against a target via `New-PSSession`. The orchestrator script passes the target list, credentials, and module path into job script blocks using `$using:` or `-ArgumentList`. The patterns in this section are not background. They are the architecture.

❓ Check your understanding

What's the difference between `Invoke-Command -ComputerName 'System-1'` (no session) and `Invoke-Command -Session $Session` (with session), and when does the difference actually matter?

3.3 - Authentication, Delegation, and the Double-Hop Problem

Why this matters to an operator: The double-hop problem is one of the most common "mysterious" failures in PowerShell remoting. It looks like a permissions issue but is not. Diagnosing it correctly the first time saves hours. In a domain environment, which the CVTE range now is, this problem is more relevant, not less.

The double-hop problem occurs when credentials used for the first remote connection are not automatically delegated to a second resource from inside that remote session.

What it looks like

```
# Single hop -- works:
Ops ---> System-1   (Kerberos auth, ticket presented directly)

# Double hop -- fails:
Ops ---> System-1 ---> Depot
                ^
                | Credentials NOT delegated past System-1
                | by default. Access to Depot fails.
```

Operationally, this means a script which works when you run it interactively on Ops will fail when you try to run the same logic from System-1 against Depot. The failure looks like a permissions problem that is not really a permissions problem. Your credentials simply did not travel with the second hop.

Why it happens

Kerberos by default does not delegate credentials from one system to another. When Ops authenticates to System-1, System-1 has a token representing the Ops user, but System-1 cannot use that token to authenticate to a third resource (Depot) on the user's behalf, because Kerberos was designed that way for security.

This is not a configuration mistake. It is the protocol's default behavior. Solving the double-hop problem requires explicitly granting one of the supported delegation patterns.

What to do about it

Two practical options exist for solving the double-hop. Both are real, both are documented, and both have tradeoffs:

- **CredSSP (Credential Security Support Provider).** The simpler option. The user's credentials are forwarded explicitly to the intermediate machine, which can then use them to

authenticate onward. Requires configuration on both ends. Works well for ad-hoc operator use. Has security tradeoffs because the intermediate machine receives the user's full credential.

- **Resource-Based Kerberos Constrained Delegation (RBKCD).** The production option. The target resource (Depot) is configured to allow specific intermediate machines (System-1) to authenticate on behalf of specific users. Configured in Active Directory, no client-side setup. Limits delegation to specific resource paths. Preferred in environments where credential exposure matters.

For ad-hoc work and testing in this course, CredSSP is the simpler path. For production environments after this course, RBKCD is the right answer.

CredSSP quick reference

```
# On Ops (the originator):
Enable-WSManCredSSP -Role 'Client' -DelegateComputer 'System-1'

# On System-1 (the intermediate hop):
Enable-WSManCredSSP -Role 'Server'

# Then run the remoting call with -Authentication CredSSP:
Invoke-Command -ComputerName 'System-1' -Authentication CredSSP -Credential $Cred -
ScriptBlock {
    Get-ChildItem '\\Depot\Reports'
}
```

The two `Enable-WSManCredSSP` calls (one on the client, one on the server) configure the trust. The `-Authentication CredSSP` flag on the `Invoke-Command` call tells PowerShell to use it.

⚠️ CredSSP exposes credentials to the intermediate machine

When you authenticate via CredSSP to System-1, System-1 receives your full credential, not just a token. If System-1 is compromised, your credential is compromised. Use CredSSP for course work and ad-hoc admin tasks, not for high-value production accounts. RBKCD is the production-grade answer because it does not expose credentials this way.

🔧 Try it

Without CredSSP configured, run `Invoke-Command -ComputerName 'System-1' -ScriptBlock { Get-ChildItem '\\Depot\C$\Tmp' } -Credential $Cred` . Note the exact error. Then enable CredSSP per the example above and run the same command with `-Authentication CredSSP` . The difference between the two error messages is the lesson.

✅ The minimum operator takeaway

You do not need to know the deep mechanics of Kerberos or RBKCD to function in this course. You do need to know: (1) the double-hop problem exists, (2) it shows up as a permission

error that is not really a permission error, (3) CredSSP is the quick fix, (4) RBKCD is the right fix in production. The names matter so you can Google when you hit it in the field.

? Check your understanding

Your script works when you run it interactively from Ops, but fails when you put the same code inside a job that runs on System-1 against Depot. Before assuming a permissions problem on Depot, what's the first thing you should check?

3.4 - Common Remote Execution Issues

Why this matters to an operator: Most remote-execution failures fall into a small set of recognizable patterns. A symptom-to-cause table is not just convenient. It is the difference between diagnosing a failure in 90 seconds and chasing it for an hour. Memorize this table or know where to find it.

Remote execution introduces several recurring failure modes. Most are quick to diagnose if you know what to look at first.

Symptom	Likely cause	First check
WinRM cannot connect	WinRM not enabled or firewall blocking	`Test-WSMan <target>`
Access denied	Wrong credential format, bad password, or insufficient permissions	See credential format callout below
Command not found remotely	Module not installed on target	Run `Get-Command` inside remote session
Path not found	Path exists on Ops but not on target	`Invoke-Command { Test-Path ... }`
Silently wrong results	Execution context differs	Run `whoami` inside remote session
Works locally, fails in job	Variable scope: parent vars not inherited	Use `-ArgumentList` or `$using:`

First-line diagnostic: connectivity and identity check

```
# Quick connectivity and identity check
$Cred = Get-Credential
Invoke-Command -ComputerName 'System-2' -Credential $Cred -ScriptBlock {
    whoami
    hostname
}
```

This three-line block answers two questions immediately: (1) can you reach the target at all, and (2) are you authenticated as the user you think you are. If either `whoami` or `hostname` returns the wrong value, the problem is upstream of whatever script you were trying to run.

⚠️ Common mistake: wrong credential format

In this domain-based range, enter credentials in one of two formats:

- **NetBIOS:** `[DOMAIN]\\[user]` . For example, if the domain is named `CVTELAB` and the user is `student01` , enter `CVTELAB\\student01` .
- **UPN:** `[user]@[domain.fqdn]` . For example, `student01@cvtelab.local` .

If you enter just `student01` with no domain qualifier, Windows resolves it against the local account database on your workstation, not the domain. You will either authenticate as a local account (wrong identity) or fail outright.

Both NetBIOS and UPN formats work with `Get-Credential` , `New-PSSession -Credential` , and `Invoke-Command -Credential` . If one format fails unexpectedly, try the other before assuming a permissions problem.

🔧 Try it

When a remote command fails, run the three-line check above against the same target before debugging the script. Half the time, the problem is identity. The other half, it's connectivity. Neither is a script problem, and recognizing that saves you from rewriting code that was never broken.

✅ Why this is a chapter centerpiece

Section 3.4 will be the page you return to most often during the course. The capstone exercises in Chapter 6 hit at least three of these symptoms during normal development. Bookmark this section. The patterns repeat.

❓ Check your understanding

You see "Access denied" when running a remote command. What are the three different things that could be true, and how do you tell them apart in under 60 seconds?

3.5 - Language Modes

Why this matters to an operator: When a script silently refuses to run on a hardened host (no error, no clear failure, just nothing happens), language mode is one of the first things to check. It is also a defender's tool: defenders use language modes to limit what PowerShell can do, even to legitimate users. Knowing it exists is half the battle.

PowerShell language modes control what a session is allowed to do. They are a security mechanism that restricts script capabilities, typically configured via Group Policy, AppLocker, or Windows Defender Application Control (WDAC).

The four language modes

Language Mode	Behavior
FullLanguage	All PowerShell features available. The default on most workstations.
RestrictedLanguage	Limited to a small set of cmdlets. Used by some constrained UI environments.
NoLanguage	PowerShell engine effectively disabled. Rare. Only specific cmdlets execute.
ConstrainedLanguage	The interesting one. Most cmdlets work, but type acceleration, .NET method calls, and other "advanced" features are blocked. Common in hardened environments and AppLocker scenarios.

Checking your current language mode

```
$ExecutionContext.SessionState.LanguageMode
```

If this returns `FullLanguage`, you are unrestricted. If it returns anything else, certain operations may silently fail or throw "not in this language mode" errors. `ConstrainedLanguage` is the one you are most likely to encounter on a hardened host.

> ⚠️ **ConstrainedLanguage failures look like script bugs**
>
> In ConstrainedLanguage, calls like `[System.IO.Path]::Combine(...)` or `[datetime]::Now` (the .NET patterns from Chapter 2) will fail with "Cannot invoke method. Method invocation is supported only on core types in this language mode." The script logic is fine. The host is restricting it. If you see that error message, your problem is host policy, not your code.

🔧 Try it

Run the language mode check on your Ops workstation. Then check it inside a remote session: `Invoke-Command -ComputerName 'System-1' -ScriptBlock { $ExecutionContext.SessionState.LanguageMode }` . The two values may differ depending on how the systems are configured. This is exactly the kind of context check from Section 3.1.

✅ Why this matters defensively

Adversaries who land on a host often try to run PowerShell to expand their access. ConstrainedLanguage is one of the controls defenders use to make that harder. Even if the adversary has full local admin, they cannot use the more powerful PowerShell features without bypassing the policy. As an operator, you should know this exists, both because you may write scripts that need to work in constrained environments, and because you should not be surprised when a defender's environment imposes it.

❓ Check your understanding

Your script runs fine on Ops. On a customer's hardened workstation, parts of it silently do nothing. Before assuming a permissions issue, what's the single command that tells you whether language mode is the problem?

3.6 - PowerShell Abuse and Defensive Context

Why this matters to an operator: PowerShell is one of the most heavily abused administrative tools in modern Windows environments. As an operator, you write scripts that look, to a defender, a lot like what an attacker writes. Knowing how PowerShell is abused, and what defenders watch for, makes you a better script author and a better teammate to defenders.

This section is not a threat-hunting course. The goal is awareness: know the patterns, know the controls, know that your scripts run inside an environment where both adversaries and defenders are paying attention to PowerShell.

How PowerShell is abused

Abuse pattern	What it looks like	Why adversaries use it
Encoded commands	`powershell -EncodedCommand <base64>`	Bypass logging, obscure intent, bypass simple string-match detections
Download cradles	Network calls inside script blocks (e.g., `Invoke-Expression` of a remote URL)	Pull and execute payloads without writing them to disk
Living-off-the-land	Native PowerShell cmdlets used for reconnaissance or lateral movement	Avoid bringing in custom tools that AV would flag
Constrained Language bypass attempts	Code that tries to escape restricted language mode	Re-enable advanced features on a host where they were disabled

Two MITRE ATT&CK techniques cover most of what's above:

- **T1059.001 Command and Scripting Interpreter: PowerShell.** The umbrella technique for any adversary use of PowerShell.
- **T1027 Obfuscated Files or Information.** Encoded commands and string obfuscation fall under this.

How defenders watch for abuse

Four logging and inspection controls produce most of the operational signal defenders use:

Control	What it does	Operational signal
Script Block Logging	Logs every script block that PowerShell executes, including dynamically generated and decoded code	`Microsoft-Windows-PowerShell/Operational, Event ID 4104`
Module Logging	Logs cmdlet invocations from specified modules	`Event ID 4103`
Transcription	Captures full session input and output to a file	Configured location, one file per session
AMSI	Inspects script content at the runtime layer, after deobfuscation, before execution	Triggered detections appear in Defender / EDR alerts

Script Block Logging (Event ID 4104) is the most important of these for an operator to know. When enabled, it captures the actual code PowerShell executed, including code that was originally encoded or built dynamically. This means "clever" obfuscation does not hide the script from the log, only delays the visibility.

⚠️ Your scripts produce logs too

Script Block Logging captures everything: your legitimate course scripts, your capstone, your test runs. This is normal and expected. The point is not that logs are bad. The point is that you should never write a script assuming it will not be observed. If your script does something that would look suspicious in 4104 logs, the answer is to communicate with the defender team, not to obfuscate.

🔧 Try it

On Ops, open Event Viewer and navigate to `Microsoft-Windows-PowerShell/Operational`. Filter for Event ID 4104. Run any PowerShell command (for example, the `Test-WSMan` from earlier) and watch the new log entry appear. The full script block is in the message body. This is what every defender sees, in real time, when you run scripts in their environment.

✅ The operator mindset

You are not an adversary, and your scripts are not an attack. But the techniques you use (remoting, credential handling, parallel execution, structured output) are also the techniques an adversary uses for similar mechanical reasons. Writing your scripts cleanly, with clear naming and predictable structure, makes the defender's job easier. Obscuring or compressing your code for "efficiency" in a contested environment makes you look like the wrong thing.

❓ Check your understanding

If you're operating on a host where Script Block Logging is enabled, and your script uses base64-encoded commands for legitimate reasons (an embedded credential rotation tool, say), what should your operational behavior be? Hint: the answer involves communication, not technique.

3.7 - DSC, APIs, and Platform Integration

Why this matters to an operator: Real environments are not just Windows servers and PowerShell remoting. They include configuration management, REST APIs, cloud platforms, and security tools that all expose interfaces. Knowing where PowerShell fits in that broader picture lets you reach for it appropriately, and avoid using it where another tool is the right answer.

This section is a survey, not a deep-dive. The goal is to know what these things are and roughly how PowerShell touches them, so you recognize them in the field.

DSC: Desired State Configuration

DSC is a declarative configuration management system. Instead of writing a script that says "install IIS," you write a configuration that says "IIS should be installed." DSC's job is to make the system match that desired state, and to re-converge if drift occurs.

```
Configuration WebServerSetup {
    Node 'System-1' {
        WindowsFeature IIS {
            Ensure = 'Present'
            Name   = 'Web-Server'
        }
    }
}
```

DSC is most useful for fleet-wide configuration management, keeping hundreds or thousands of systems in a known state. For one-off scripts targeting a handful of systems, DSC is overkill. The capstone uses imperative remoting (Chapter 5), not DSC.

REST APIs

Most modern platforms expose REST APIs. PowerShell's `Invoke-RestMethod` cmdlet returns parsed JSON or XML directly as PowerShell objects, which means API integration follows the same patterns as the rest of your scripting:

```
$Headers = @{ Authorization = "Bearer $Token" }
$Response = Invoke-RestMethod -Uri 'https://api.example.com/v1/devices' -Headers $Headers
$Response.devices | Where-Object Status -eq 'Active'
```

Authentication is the variable part. Some APIs use bearer tokens (shown above), some use API keys in headers, some use mTLS. The mechanics differ; the pattern is the same: set up authentication, call the endpoint, work with the returned objects.

> 🔧 **Try it**
> If you have access to a public API that does not require authentication, try `Invoke-RestMethod 'https://jsonplaceholder.typicode.com/users'`. The result is an array of 10 user objects with named properties: name, email, address, company, and more. Pipe it to `Where-Object`, `Select-Object`, or `Sort-Object` just like any other PowerShell object. The same patterns from Chapter 1 apply, even though the data came over the network.

Other platform integrations worth knowing

- **Active Directory.** The `ActiveDirectory` module exposes AD operations as cmdlets. Useful for user/group/computer-object work, less useful for raw LDAP queries.
- **Azure / cloud.** Each major cloud provides PowerShell modules (`Az.*`, `AWS.Tools.*`, etc.) that wrap their REST APIs. The patterns in this section apply, with provider-specific authentication.
- **Security tools.** Many EDR, SIEM, and identity tools expose PowerShell modules or REST APIs. Operationally, scripted queries against these tools are how operators automate response and investigation work.

> ✅ **Course scope reminder**
> The capstone does not require DSC, REST API integration, or cloud module usage. This section is for operational awareness, so when you encounter these in the field, you know where they fit. If you find yourself using DSC inside the capstone, you have probably overcomplicated the solution.

> ❓ **Check your understanding**
> When would you reach for DSC over a regular PowerShell remoting script? Give one operational scenario where DSC genuinely is the right answer.

3.8 - Chapter 3 Exercise: Remote Connectivity Validator

Why this matters to an operator: This exercise integrates everything from Chapter 3: WinRM testing, credentials, persistent sessions, variable scoping, error handling, structured output, and cleanup. It is the most realistic small task you will write in the course up to this point. If this works cleanly, you are ready for Chapter 4.

> 💡 **Scenario**
> You need to validate connectivity and authentication to a list of remote targets before kicking off a larger script run. The validator should reach each target, open a session, confirm identity, and return a structured per-target result. Failures must be captured cleanly. The validator should never crash the calling script.

Exercise Requirements

Build a function called `Test-RemoteConnectivity` that:

- Accepts a list of computer names and a credential as parameters
- Tests WinRM reachability against each target
- Opens a `PSSession` to each reachable target
- Inside each session, captures `whoami` (the authenticated identity) and `hostname` (confirmation of which target you actually reached)
- Returns one `[pscustomobject]` per target with: `ComputerName`, `Reachable`, `AuthenticatedAs`, `Hostname`, and `Error`
- Closes every session it opens, even if errors occur

Exercise Skeleton: Test-RemoteConnectivity.ps1

```
function Test-RemoteConnectivity {
    param(
        [string[]]$ComputerName = # ,
        [pscredential]$Credential = #
    )

    # 1. For each target, test WinRM connectivity
    #    Hint: Test-WSMan, with -ErrorAction SilentlyContinue

    # 2. For reachable targets, open a PSSession
    #    Hint: New-PSSession with -Credential

    # 3. Inside each session, run whoami and hostname
    #    Hint: Invoke-Command -Session, $using: if needed

    # 4. Return one [pscustomobject] per target with:
    #    ComputerName, Reachable, AuthenticatedAs, Hostname, Error

    # 5. Always close sessions, even if the script errors out
    #    Hint: try/finally with Remove-PSSession
}
```

Success criteria

- The function returns one row per input target, regardless of whether each target succeeded or failed
- The same property names appear on every row (success and failure paths return identical shapes)
- No orphaned sessions remain after the function returns. Verify with `Get-PSSession`
- The function does not throw when a target is unreachable; it returns a row with `Reachable = $false` and the error captured in the `Error` property

⚠️ Common mistakes on this exercise

Three things consistently fail this exercise:

- **Different shapes on success vs failure paths.** If your success path returns `ComputerName, Reachable, AuthenticatedAs, Hostname, Error` but your failure path returns just `ComputerName, Error`, downstream consumers cannot work with the output cleanly.
- **Orphaned sessions.** If you open a session inside the loop and only close it on success, failures leave the session open. Use `try/finally` or `Get-PSSession | Remove-PSSession` in cleanup.
- **Forgetting `$using:`.** If you reference local variables inside `Invoke-Command -ScriptBlock { ... }` without `$using:`, the remote session sees nothing.

> ✅ **Self-check before you consider this done**
> (1) Run the function with all four targets reachable. Does it return four success rows?
> (2) Run it with one target name deliberately wrong. Does it return three success rows and one failure row, all with the same shape?
> (3) After the function returns, run `Get-PSSession` . Is it empty?
> If all three pass, you are done. If any fail, the failure points to which Chapter 3 concept needs more attention.

Chapter 4: Performance and Parallel Execution

> 💡 **Chapter Purpose**
> This chapter develops the performance habits and job-management patterns that make large-scale execution practical. The focus is not speed for its own sake. The focus is understanding when work can be reduced, when concurrency is appropriate, and how to coordinate parallel tasks from Ops without losing control of inputs, imports, or results.

By the end of Chapter 4, performance should feel like a design consideration rather than a cleanup task. A script that works once is useful. A script that works across multiple targets, returns structured results, and finishes in a predictable amount of time is far more valuable.

This chapter is also the bridge to the orchestrator. The capstone does not ask for every possible concurrency technique in PowerShell. It asks for a clean, reliable pattern that can coordinate work across the course targets from Ops.

System	Role in this chapter
Ops	Execution host where jobs are created, monitored, and collected
System-1	Remote target
System-2	Remote target
System-3	Remote target
Depot	Remote target and report repository

Learning Objectives

- Establish performance baselines using `Measure-Command` . **(K1)**
- Identify and remove unnecessary work before introducing concurrency. **(K2)**
- Use jobs to run independent units of work in parallel. **(K2)**
- Pass values, imports, and modules into jobs deliberately. **(K2)**
- Apply the canonical course pattern to coordinate work across multiple targets. **(K2)**

Chapter Map

Section	Focus
4.1	Measuring performance
4.2	Improving efficiency before parallelizing

Section	Focus
4.3	Jobs and parallel execution
4.4	Scope, imports, and inputs in jobs
4.5	Parallel patterns for this course
4.6	Chapter review and guided practice
4.7	Chapter exercise: Parallel Performance Auditor

> ✅ **What matters most**
> Three things carry forward to the capstone: measuring before optimizing, understanding that jobs run in their own session context, and the one-job-per-target pattern in 4.5. Internalize those three and the capstone's parallel orchestrator becomes a small extension of work you have already done.

4.1 - Measuring Performance

Why this matters to an operator: Optimizing without measuring is guessing. The script that "feels slow" might actually be fast except for one bad call. The script that "feels fine" might be wasting twenty seconds you didn't notice. A measurement turns intuition into data. That is the entire point of this section.

Performance work begins with one discipline: measure the script that actually exists before deciding how to improve it. That sounds simple, but it prevents a common mistake. A script can feel slow because it touches several targets, because it repeats the same work unnecessarily, or because it introduces overhead that is easy to miss when only one command is tested in isolation.

For this course, the most useful measurement is wall-clock time. How long the whole operation takes from start to finish. That is the number that matters when Ops must coordinate work across System-1, System-2, System-3, and Depot.

Using Measure-Command well

`Measure-Command` wraps a block of code, runs it, and returns timing data. It does not change the script. It simply gives the script a stopwatch.

```
$Targets = 'System-1', 'System-2', 'System-3', 'Depot'

$Timing = Measure-Command {
    foreach ($ComputerName in $Targets) {
        Invoke-Command -ComputerName $ComputerName -ScriptBlock {
            Get-Service -Name WinRM |
            Select-Object MachineName, Status, StartType
        }
    }
}

$Timing.TotalSeconds
```

This example measures the total time required to query the same service on four targets from Ops. The returned value is not the service data. It is a `TimeSpan` object that describes how long the operation took.

```
$Timing.GetType().FullName
$Timing | Select-Object Days, Hours, Minutes, Seconds, Milliseconds, TotalSeconds
```

That extra inspection matters. Looking only at a formatted value can hide useful properties. The object tells the full story.

> 🔧 **Try it**
> Run the timing block above on your Ops workstation. Note the value of `.TotalSeconds`. Now pipe `$Timing` to `Get-Member`. The output reveals every property and method on a `TimeSpan` object. This is the same Get-Member discipline from Chapter 1, applied to a measurement object.

What a measurement does and does not mean

One timing result is informative, but it is not final. Remote work can vary based on connection state, current load, caching, and network conditions. The right habit is to compare runs under the same conditions and look for patterns rather than assuming one number is absolute.

```
1..3 | ForEach-Object {
    Measure-Command {
        foreach ($ComputerName in $Targets) {
            Invoke-Command -ComputerName $ComputerName -ScriptBlock {
                Get-Service -Name WinRM | Select-Object MachineName, Status
            }
        }
    } | Select-Object TotalSeconds
}
```

Repeated measurements help answer a more useful question: is the script consistently slow, or was one run unusual?

> ⚠️ **Reading the result honestly**
> Use a measurement to guide the next design choice. A timing result by itself does not prove that jobs are required. It only shows how long the current approach takes. Before adding concurrency, ask whether the slow part is actually doing necessary work.

Sequential work is often the baseline

Before any parallel pattern is introduced, it is helpful to establish a clear baseline. When a script runs sequentially, the flow is easy to understand: one target finishes, then the next begins. That predictability makes sequential execution the right place to start even when the long-term goal is concurrency.

In the next section, the goal is not to jump straight to parallel execution. The goal is to remove waste first.

✅ The capstone connection

The capstone orchestrator runs in parallel, but you will almost certainly debug it sequentially first. Establishing a baseline lets you confirm the parallel version is actually faster, not just busier. `Measure-Command` is the tool that confirms it.

❓ Check your understanding

If your sequential script takes 12 seconds against four targets, what's the theoretical fastest a parallel version could be? What's a realistic fastest? And why is the answer not the same?

4.2 - Improving Efficiency Before Parallelizing

Why this matters to an operator: Throwing concurrency at a slow script often makes it busier without making it faster. The biggest performance wins almost always come from doing less work, not from doing the same work in parallel. This section is the discipline that gets applied before reaching for jobs.

Parallel execution is powerful, but it does not fix inefficient logic. A script that repeats unnecessary work will usually remain wasteful even when that work is spread across several jobs. The cleanest performance gains often come from improving the design before introducing concurrency.

Reduce work before adding workers

Several common issues increase runtime without adding value:

- Reading the same data repeatedly inside a loop
- Filtering late instead of early
- Building remote calls around information that could have been prepared once on Ops
- Collecting screen output instead of returning usable objects

The general rule is simple: perform shared setup once, then reuse it.

```
# Less efficient: repeated setup around the real task
foreach ($ComputerName in $Targets) {
    $SelectedService = Get-Service | Where-Object Name -eq 'WinRM'
    Invoke-Command -ComputerName $ComputerName -ScriptBlock {
        Get-Service -Name WinRM | Select-Object MachineName, Status
    }
}

# More efficient: define the intent once, then reuse it
$ServiceName = 'WinRM'
foreach ($ComputerName in $Targets) {
    Invoke-Command -ComputerName $ComputerName -ScriptBlock {
        param($ServiceName)
        Get-Service -Name $ServiceName | Select-Object MachineName, Status
    } -ArgumentList $ServiceName
}
```

The improvement is not dramatic because the example is small, but the principle scales. Repeating work that can be done once is one of the easiest ways to slow down a larger script.

Filter and shape early

Another useful habit is to reduce the amount of data carried forward. If only a few properties matter, select them. If only a subset of targets should be processed, determine that before the heavy work begins.

```
$Targets = 'System-1', 'System-2', 'System-3', 'Depot'
$TargetsToQuery = $Targets | Where-Object { $_ -ne 'Depot' }

foreach ($ComputerName in $TargetsToQuery) {
    Invoke-Command -ComputerName $ComputerName -ScriptBlock {
        Get-Service -Name WinRM |
        Select-Object MachineName, Status, StartType
    }
}
```

That approach makes the intent clearer and reduces unnecessary work. The same idea applies to objects: return the properties that matter, not every property available by default.

> 🔧 **Try it**
> Take any script you have written and look at the loops in it. For each loop, ask one question: is there anything inside the loop that could happen once before the loop instead? If the answer is yes, move that work outside. Time the script before and after with `Measure-Command`. The difference is usually larger than expected.

Efficiency improves readability too

Well-structured scripts are usually easier to read. Inputs are defined once. Filters are visible. Remote actions are deliberate. Performance and readability often improve together because both benefit from clear design.

> ✅ **Checkpoint**
> Before using jobs, confirm that the script is already doing only the work that needs to be done. Parallelizing waste is still waste. The capstone rubric does not reward clever concurrency over a clean design.

> ❓ **Check your understanding**
> A teammate's script takes 30 seconds to query four targets. They want to make it faster by switching to jobs. Before agreeing, what's the one diagnostic question you should ask first?

4.3 - Jobs and Parallel Execution

Why this matters to an operator: Jobs are how PowerShell runs work in the background. Once you understand the lifecycle, the cmdlets are easy. The hard part is recognizing when jobs help, when they don't, and when they introduce more problems than they solve. This section establishes the lifecycle. Section 4.4 establishes the gotchas.

Once the script has a clean sequential design, the next question is whether the units of work are independent. If each target can be processed without depending on the result from another target, the task is a good candidate for parallel execution.

In PowerShell, a job is a background task that runs separately from the foreground prompt. Jobs make it possible to start multiple operations, continue working, then collect the results later.

Basic job lifecycle

```
$Job = Start-Job -ScriptBlock {
    Get-Date
}

Get-Job
Wait-Job -Job $Job
Receive-Job -Job $Job
Remove-Job -Job $Job
```

That pattern is worth memorizing. Jobs are started, monitored, waited on, received, and removed. Leaving jobs behind creates clutter and makes later troubleshooting harder.

> 🔧 **Try it**
> Run the lifecycle block on your Ops workstation. Then run it again, but skip `Remove-Job`. Run `Get-Job` a few times. Notice the completed job is still listed. Now run a few more `Start-Job` calls without cleanup. After 5-10 jobs, your session is cluttered. `Get-Job | Remove-Job` clears it. This is exactly the kind of state that quietly degrades a session over the course of a day.

Applying jobs to multiple targets

The more useful pattern for this course is one job per target. Ops launches the jobs. Each job performs the same kind of work against a different remote system.

```powershell
$Targets = 'System-1', 'System-2', 'System-3', 'Depot'

$Jobs = foreach ($ComputerName in $Targets) {
    Start-Job -Name $ComputerName -ArgumentList $ComputerName -ScriptBlock {
        param($ComputerName)

        Invoke-Command -ComputerName $ComputerName -ScriptBlock {
            Get-Service -Name WinRM |
            Select-Object MachineName, Status, StartType
        }
    }
}

$null = $Jobs | Wait-Job
$Results = $Jobs | Receive-Job
$Jobs | Remove-Job

$Results
```

This pattern reduces total wall-clock time when the work is truly independent and the overhead of running jobs is justified. The gain is usually seen at the whole-run level, not in the speed of an individual query.

When jobs help and when they do not

Jobs are not automatically faster. Very small or very short tasks may complete so quickly that the extra overhead adds little value. Jobs are most helpful when there are several similar units of work that can be started and collected independently.

> ✅ **Good candidate for jobs**
> Processing the same action across System-1, System-2, System-3, and Depot is a good fit because each target can be handled independently. The total runtime drops from the sum of all targets to roughly the slowest single target, plus job overhead.

> ⚠️ **Bad candidate for jobs**
> If the operations depend on each other (output from one feeds input to the next), or if each operation is so fast the job overhead exceeds the work, jobs make things slower, not faster. Measure first.

> ❓ **Check your understanding**
> You have a script that runs four ten-second queries sequentially. You convert it to four parallel jobs. The total runtime drops from 40 seconds to 11 seconds. Where did the extra second come from?

4.4 - Scope, Imports, and Inputs in Jobs

Why this matters to an operator: The single most common job failure in this course is a function or variable that "should" be available but isn't. The cause is almost always scope. A job runs in its own session. Anything the job needs has to be passed in or imported in. If you remember nothing else from this section, remember that.

The most important detail about jobs is that they run in a separate session context. A function available in the current console may not exist inside the job. A variable visible in the current script may not be present inside the job. A module imported earlier may need to be imported again inside the job.

Most job failures in this course come from that separation. The command works in the console, then fails in the background because the job cannot see the same state.

Passing values into a job

Use parameters and arguments to pass required values into the job script block deliberately.

```
$ComputerName = 'System-1'
$ServiceName = 'WinRM'

Start-Job -ArgumentList $ComputerName, $ServiceName -ScriptBlock {
    param($ComputerName, $ServiceName)

    Invoke-Command -ComputerName $ComputerName -ScriptBlock {
        param($ServiceName)
        Get-Service -Name $ServiceName | Select-Object MachineName, Status
    } -ArgumentList $ServiceName
}
```

That design makes the inputs obvious and avoids hidden dependencies.

> ⚠️ **Two scope boundaries, not one**
>
> Notice the example above has two `param()` blocks, two `-ArgumentList` calls. Outer one passes from your console into the job. Inner one passes from the job into the remote session. Both are real boundaries. Skip either and the variable disappears.

Importing a module inside the job

If a job needs a function stored in a module, import that module inside the job. Do not assume that importing it in the parent session is enough.

```powershell
$ModulePath = 'C:\Tools\JobExamples.psm1'
$ComputerName = 'System-1'

Start-Job -ArgumentList $ModulePath, $ComputerName -ScriptBlock {
    param($ModulePath, $ComputerName)

    Import-Module $ModulePath -Force
    Get-RemoteWinRMStatus -ComputerName $ComputerName
}
```

The specific function name will vary later in the course, but the pattern remains the same: pass the module path, import it inside the job, then call the function.

> 🔧 **Try it**
> Define a small function in your console, then call `Start-Job -ScriptBlock { YourFunctionName }`. Wait for the job, then receive it. The job fails: the function is not defined inside the job's session. Now wrap the function inside the script block instead, and watch it work. This is the scope boundary in action.

Returning objects instead of screen-only text

Jobs are much easier to work with when they return structured objects. A formatted string may look readable, but it is harder to sort, filter, export, and validate later.

```powershell
Start-Job -ArgumentList $ComputerName -ScriptBlock {
    param($ComputerName)

    [pscustomobject]@{
        ComputerName = $ComputerName
        Reachable    = $true
        Timestamp    = Get-Date
    }
}
```

That result can be collected, combined, and exported with very little cleanup.

> ✅ **Troubleshooting order**
> When a job fails unexpectedly, follow this order: first verify the inputs (did the values arrive in the job?), then verify the module or function availability inside the job, then inspect the returned errors before changing the larger design. Most job failures resolve at one of those three checks.

> ❓ **Check your understanding**

Why do `Start-Job` patterns commonly use `-ArgumentList` instead of just referencing the outer variable directly? What's actually happening at the boundary?

4.5 - Parallel Patterns for This Course

Why this matters to an operator: The patterns above can be combined many ways. This section narrows to the one pattern this course standardizes on. The capstone is built around this exact shape. If your Chapter 4 work matches this pattern, your Chapter 5 and 6 work will fall into place naturally.

At this point the chapter narrows to one course pattern: Ops defines the targets, starts one job per target, waits, receives, and cleans up. That is the parallel pattern that matters most for the orchestrator.

Canonical course pattern

```
$Targets = 'System-1', 'System-2', 'System-3', 'Depot'
$ModulePath = 'C:\Tools\JobExamples.psm1'

$Jobs = foreach ($ComputerName in $Targets) {
    Start-Job -Name $ComputerName -ArgumentList $ModulePath, $ComputerName -ScriptBlock {
        param($ModulePath, $ComputerName)

        Import-Module $ModulePath -Force

        Get-RemoteWinRMStatus -ComputerName $ComputerName
    }
}

$null = $Jobs | Wait-Job
$Results = $Jobs | Receive-Job
$Jobs | Remove-Job

$Results
```

Even though the specific function later changes, the orchestration logic stays familiar. The calling script coordinates the run. The module does the task-specific work. That division keeps the design easier to read and easier to test.

Why this pattern works well for the course

- The workflow remains centralized on Ops
- Each target becomes one independent unit of work
- Results are gathered into one collection after completion
- Module-based logic can be reused without rewriting the orchestrator

This is also where discipline matters. The pattern is simple on purpose. It is better to use one clean, dependable pattern well than to switch between several valid but inconsistent approaches.

> ✅ **Design standard**
> The orchestrator coordinates the run. The module performs the target-specific work. Keep this division clean and the rest of the course gets easier.

Sequential and parallel designs can coexist

Parallel execution is useful when the work benefits from it. Sequential execution is still valuable for initial testing, troubleshooting, or small-scale validation. A dependable workflow often starts sequentially, proves the logic, and then adopts the parallel wrapper once the task logic is stable.

A note on ForEach-Object -Parallel

PowerShell 7 added a parallel mode to `ForEach-Object` that runs script blocks concurrently without explicit job management:

```
$Results = $Targets | ForEach-Object -Parallel {
    Invoke-Command -ComputerName $_ -ScriptBlock {
        Get-Service -Name WinRM
    }
}
```

This is a real and valid pattern. It does not produce the same return shape as `Start-Job`, the runspace lifecycle is different, and `$using:` is the only way to pass variables in (no `-ArgumentList` option). For one-off operator scripting it is often simpler than jobs. For the capstone, this course uses jobs because the capstone rubric is built around `Start-Job` / `Wait-Job` / `Receive-Job` mechanics, and jobs handle module imports more cleanly than parallel runspaces do.

> 💡 **When to reach for ForEach-Object -Parallel in the field**
> Quick one-off operations across many items where you don't need module imports, persistent sessions, or a structured wait/receive pattern. `ForEach-Object -Parallel` is faster to write and has less ceremony. For anything you would build into a tool, jobs are usually the right answer because they give you more control over the lifecycle.

> 🔧 **Try it**
> Run the canonical pattern above. Now run the same task with `ForEach-Object -Parallel`. Time both with `Measure-Command`. Note which one is faster, and notice that the output shape is different. The choice between them is not always about speed.

> ✅ **The capstone connection**
> Your capstone orchestrator will look almost exactly like the canonical pattern above, with one of three real `Get-Remote*` functions in place of `Get-RemoteWinRMStatus` and a different module name. If the canonical pattern feels automatic by the end of this chapter, the capstone orchestration is mostly already written.

4.7 - Chapter 4 Exercise: Parallel Performance Auditor

Why this matters to an operator: This exercise integrates the entire chapter into one concrete deliverable. You measure a real baseline, implement the canonical parallel pattern, and compare runtimes side by side. By the end, you have empirical evidence that parallel execution actually does what you think it does. That evidence carries forward to Chapter 5 and the capstone.

> 💡 **Scenario**
> You have been asked to audit how long a status check against the four targets takes, and to confirm whether converting it to a parallel pattern actually improves runtime. Build the auditor as a single script that runs both versions and reports the difference.

Exercise Requirements

Build a script called `Test-ParallelPerformance.ps1` that does the following:

- **Part 1: Sequential Baseline.** Use `Measure-Command` to time a standard `foreach` loop that runs `Invoke-Command` against each target in sequence, querying the `WinRM` service.
- **Part 2: Parallel Implementation.** Use `Measure-Command` to time the canonical course pattern from 4.5. Start one job per target. Wait. Receive. Remove.
- **Part 3: Reporting.** Output both timings and display the data the parallel run actually retrieved.

Exercise Skeleton: Test-ParallelPerformance.ps1

```
# Chapter 4 Exercise: Parallel Performance Auditor (Skeleton)
# Goals: Measure baseline, implement parallel jobs, and compare runtimes.

$Targets = @('System-1', 'System-2', 'System-3', 'Depot')
$ServiceToCheck = 'WinRM'

# --- Part 1: Sequential Baseline ---
# Use Measure-Command to time a standard foreach loop
$SequentialTime = Measure-Command {
    # Implementation: Loop through targets and run Invoke-Command sequentially
}

# --- Part 2: Parallel Implementation ---
# Use Measure-Command to time the job-based approach
$ParallelTime = Measure-Command {
    # 1. Start one job per target using the Canonical Course Pattern
    # Hint: Pass $ServiceToCheck into -ArgumentList
    $Jobs = foreach ($Target in $Targets) {
        # Start-Job implementation
    }

    # 2. Wait for all jobs to complete

    # 3. Receive results into a variable

    # 4. Remove jobs to clean the session
}

# --- Part 3: Reporting ---
# Output the timing comparison and the retrieved data
Write-Host "Sequential Run: $($SequentialTime.TotalSeconds) seconds"
Write-Host "Parallel Run:   $($ParallelTime.TotalSeconds) seconds"

# Display the received results table
```

Success criteria

- Both `$SequentialTime` and `$ParallelTime` are populated with valid `TimeSpan` objects after the script runs
- The parallel run completes faster than the sequential run (typically by 50% or more, but the exact ratio varies by network conditions)
- The parallel run returns retrieved data, not just timing information
- No orphaned jobs remain after the script completes (verify with `Get-PSSession` returning empty, and `Get-Job` returning empty)
- The script is re-runnable: running it twice in the same session produces consistent behavior with no leftover state

⚠️ Common gotchas on this exercise

Three things consistently fail this exercise:

- **Variable scope inside the job.** If you reference `$ServiceToCheck` inside the job's script block without passing it via `-ArgumentList` and a `param()` block, the job sees a null value. The job will not error - it will quietly use null and return nothing useful.
- **Forgetting to remove jobs.** Without the `Remove-Job` step, repeated runs accumulate completed jobs in your session. Eventually `Get-Job` becomes a wall of stale entries.
- **Comparing different work.** If your sequential block does more or less work than your parallel block (different cmdlets, different filters, different return data), the timing comparison is not meaningful. Both blocks should perform the same logical task.

💡 Why Write-Host is acceptable here

Chapter 1.7 strongly favored `[pscustomobject]` over `Write-Host` for script output. The reporting section of this exercise uses `Write-Host` for the timing summary, which is intentional. The timings are operator-facing diagnostic information, not data being passed forward in a pipeline. The retrieved results table at the end is the structured output. This is the right time to use `Write-Host` - when the audience for that specific line is genuinely a human eye.

✅ Self-check before you consider this done

(1) Run the script. Does it produce a sequential timing, a parallel timing, and a results table?
(2) Compare the timings. Is the parallel version faster?
(3) Run `Get-Job` after the script finishes. Is it empty?
(4) Run the script a second time without restarting your session. Does it behave the same as the first run?
If all four are yes, you are done. If any is no, the failure points to which Chapter 4 concept needs more attention before Chapter 5.

🔧 Optional extension

Once the basic auditor works, try running it three times in a row and capturing the runtime values. The variation between runs tells you something the single number does not - whether the network is consistent, whether one target is slower than the others, and whether the parallel speedup is reliable or just lucky. This is the kind of analysis the capstone implicitly tests for.

4.6 - Chapter Review and Guided Practice

Why this matters to an operator: Section 4.6 is the readiness check before the formal exercise in 4.7. If any of the patterns below feel unfamiliar, now is the time to revisit the section that introduced them. The exercise that follows assumes these are automatic.

Chapter 4 turned performance into a practical design habit. Measure the current approach. Remove avoidable waste. Introduce jobs only when the work is independent. Pass inputs deliberately. Import what the job needs. Return structured results. Clean up when the work is complete.

Key ideas to carry forward

Habit	Section
Use `Measure-Command` to establish a real baseline before changing the design	4.1
Reduce unnecessary work before parallelizing	4.2
Use jobs when multiple targets can be processed independently	4.3
Remember that jobs run in a separate context and may need their own imports and inputs	4.4
Return objects so the orchestrator can evaluate and reuse results later	4.4
Remove completed jobs to keep the session clean	4.3, 4.5
Use the canonical one-job-per-target pattern as the course standard	4.5

Guided practice

1. Measure a sequential query from Ops against all four targets and record the total runtime.
2. Build a one-job-per-target version of the same task and compare the total runtime.
3. Modify the job so that the service name is passed in as an argument instead of being hardcoded.
4. Create a small `[pscustomobject]` inside the job and return it through `Receive-Job`.
5. Deliberately leave jobs uncleared once, inspect them with `Get-Job`, then clean them up properly.

Review questions

1. Why is `Measure-Command` useful before introducing jobs?
2. What kinds of tasks are good candidates for one-job-per-target execution?

3. Why can a function work in the console but fail inside a job?
4. Why is returning an object usually better than returning a formatted string?
5. What are the standard steps in the job lifecycle after a set of jobs has been created?

> ✅ **Self-assessment - you are ready for 4.7 when...**
>
> - You can write the canonical one-job-per-target pattern from memory
> - You can explain why `-ArgumentList` is needed even if you have already declared the variable in the parent scope
> - You know the four-step job lifecycle (`Start-Job` , `Wait-Job` , `Receive-Job` , `Remove-Job`) and why each step matters
> - You can capture a runtime with `Measure-Command` and pull `TotalSeconds` from the result
>
> If any of those feel shaky, revisit the relevant section before starting the exercise.

> 💡 **Looking ahead**
>
> Chapter 5 builds on this pattern by shifting attention from generic job control to reusable task logic, orchestration design, and the capstone workflow. The pattern stays the same; the work inside each job becomes more interesting.

Chapter 5: Building Reusable Tools and Orchestration

> 💡 **Chapter Purpose**
> This chapter is the capstone dress rehearsal. You will build three reusable functions, package them into a module, write an orchestrator that imports the module and coordinates work across all four targets, and run that work in parallel from Ops. The functions you build here are not the capstone deliverables. They teach the same patterns using different cmdlets, so the capstone becomes a recombination instead of a from-scratch build.

By the end of this chapter, the architecture of your capstone should be visible to you. The practice module exposes a Get-* function, another Get-* function, and a Deploy-* function. The orchestrator imports the module, prompts once for credentials, and runs the functions in parallel against the targets. That shape, with three different cmdlets and one different deploy target, is exactly what your capstone submission looks like.

This is also the last chapter that introduces new material. Chapter 6 walks through the capstone build itself, but does not introduce new patterns. If the patterns in Chapter 5 feel automatic by Friday morning, the capstone is mostly assembly.

System	Role in this chapter
Ops	Where you build, import the module, and run the orchestrator
System-1	Practice target for Function 1 and Function 3
System-2	Practice target for parallel execution
System-3	Practice target for parallel execution
Depot	Practice target and report repository

Learning Objectives

- Build reusable functions that query remote systems and return structured results. **(K8, K9)**
- Use PowerShell remoting to perform remote deployment actions. **(K8)**
- Handle per-host failures without crashing the full workflow. **(K8)**
- Use a simple parallel orchestration pattern from Ops to the four remote targets. **(K8)**
- Recognize how existing function patterns can be repurposed for new tasks. **(K9)**

Chapter Map

Section	Focus
5.1	Remote query pattern
5.2	Build Function 1: remote scheduled task query
5.3	Build Function 2: remote event log query
5.4	Build Function 3: deploy remote support files
5.5	Error handling and validation
5.6	Module assembly
5.7	Orchestrator pattern
5.8	Parallel execution from Ops
5.9	Troubleshooting and cleanup
5.10	Chapter review and capstone readiness
5.11	Chapter exercise: end-to-end dry run

> ✅ **Key principle**
> The functions you build in this chapter are **not** the capstone functions. They teach the same patterns using different targets. The capstone requires you to apply these patterns to process inventory, service inventory, and Sysmon deployment. The pattern transfers; the function names, cmdlets, and deploy actions do not.

5.1 - Remote Query Pattern

Why this matters to an operator: This is the most reused pattern in the course. Every Get-* function you build for the capstone follows this exact structure. Learn it once and you can produce any number of query functions by changing what runs inside the script block. Get this pattern automatic and the rest of the chapter becomes mechanical.

Every remote query function follows the same structure. The interface is consistent (parameters in, structured object out), the error handling is consistent (try/catch with terminating errors), and the return shape is consistent (success and failure both produce the same five properties).

The pattern

1. Accept input by parameter
2. Use `Invoke-Command` with `-ErrorAction Stop`
3. On success: return `[pscustomobject]` with `Status = 'Success'`
4. On failure: return `[pscustomobject]` with `Status = 'Failed'`

Skeleton function

```
function Get-RemoteExample {
    param(
        [string]$ComputerName,
        [pscredential]$Credential
    )

    try {
        $Data = Invoke-Command -ComputerName $ComputerName -Credential $Credential -
ScriptBlock {
            # your query goes here
            hostname
        } -ErrorAction Stop

        [pscustomobject]@{
            ComputerName = $ComputerName
            Function     = 'Get-RemoteExample'
            Status       = 'Success'
            Data         = $Data
            Error        = $null
        }
    }
    catch {
        [pscustomobject]@{
            ComputerName = $ComputerName
            Function     = 'Get-RemoteExample'
            Status       = 'Failed'
            Data         = $null
            Error        = $_.Exception.Message
        }
    }
}
```

Read this carefully. The script block is a placeholder. Everything else, the parameter declaration, the try/catch wrapper, the success object shape, the failure object shape, is fixed. Learn the wrapper, then change the script block.

> 🔧 **Try it**
> Define `Get-RemoteExample` on your Ops workstation. Run it against System-1. Then run it against a target that does not exist (try `'NoSuchHost'`). Both calls return objects with the same five properties. The Status field tells you which path you took. This is what 'consistent return shape' looks like in practice.

> ⭐ **Capstone connection**
> The capstone rubric requires two Get-* functions that accept `-ComputerName`, return `[pscustomobject]`, and work against multiple remote hosts. This pattern satisfies all of those

requirements. The function names and the script block contents change for the capstone. The wrapper does not.

? Check your understanding

Why does the success path put data in the `Data` property and the failure path put `$null` there? Why not omit the property entirely on failure?

5.2 - Build Function 1: Remote Scheduled Task Query

Why this matters to an operator: Your first concrete function. The skeleton from 5.1 stays unchanged. Only the script block changes. This is the moment where the abstract pattern becomes a working tool, and where you confirm the pattern is general by applying it to a real cmdlet.

Your first practice function queries scheduled tasks from a remote target. Same pattern from 5.1, different data source.

```
function Get-RemoteTaskInfo {
    param(
        [string]$ComputerName,
        [pscredential]$Credential
    )

    try {
        $Data = Invoke-Command -ComputerName $ComputerName -Credential $Credential -
ScriptBlock {
            Get-ScheduledTask | Where-Object State -eq 'Ready' |
                Select-Object TaskName, State, TaskPath -First 10
        } -ErrorAction Stop

        [pscustomobject]@{
            ComputerName = $ComputerName
            Function     = 'Get-RemoteTaskInfo'
            Status       = 'Success'
            Data         = $Data
            Error        = $null
        }
    }
    catch {
        [pscustomobject]@{
            ComputerName = $ComputerName
            Function     = 'Get-RemoteTaskInfo'
            Status       = 'Failed'
            Data         = $null
            Error        = $_.Exception.Message
        }
    }
}
```

What changed from the skeleton

Three things, all small:

- Function name became `Get-RemoteTaskInfo`
- The `Function` property in both objects matches the new name
- The script block is now `Get-ScheduledTask | Where-Object | Select-Object` instead of `hostname`

The parameter block, the try/catch wrapper, the object shape, the error handling: all unchanged. That is the point. The skeleton is reusable.

> 🔧 **Try it**
> Run `Get-RemoteTaskInfo -ComputerName 'System-1' -Credential (Get-Credential)` on your Ops workstation. Inspect the result with `$result | Select-Object *`. Notice that `$result.Data` contains the actual scheduled tasks, while `$result.Status` tells you whether the call succeeded. This separation is intentional and matters in 5.8 when you count results from parallel runs.

> ⭐ **Capstone connection**
> For the capstone, you will build `Get-RemoteProcessInventory`. Instead of `Get-ScheduledTask`, you will use `Get-Process`. Instead of selecting TaskName, State, and TaskPath, you will select Name, Id, and Path. Everything else, the parameter block, the try/catch, the object shape, stays exactly as it is here.

> ❓ **Check your understanding**
> If `Get-ScheduledTask` returns 80 tasks but you only see 10 in the result, where in the function is that limit being applied? Could you remove it without changing the wrapper?

5.3 - Build Function 2: Remote Event Log Query

Why this matters to an operator: Function 2 confirms the pattern is general. Same skeleton, different cmdlet, different data. By the time you finish this section, the wrapper code should feel mechanical, almost reflexive. That reflexive recognition is what carries you through the capstone build.

Your second practice function queries the Windows event log on a remote target. Pattern from 5.1, different data source again.

```
function Get-RemoteEventSummary {
    param(
        [string]$ComputerName,
        [pscredential]$Credential
    )

    try {
        $Data = Invoke-Command -ComputerName $ComputerName -Credential $Credential -
ScriptBlock {
            Get-WinEvent -LogName System -MaxEvents 25 |
                Where-Object LevelDisplayName -in 'Error', 'Warning' |
                Select-Object -First 10 TimeCreated, Id, ProviderName, LevelDisplayName,
Message
        } -ErrorAction Stop

        [pscustomobject]@{
            ComputerName = $ComputerName
            Function     = 'Get-RemoteEventSummary'
            Status       = 'Success'
            Data         = $Data
            Error        = $null
        }
    }
    catch {
        [pscustomobject]@{
            ComputerName = $ComputerName
            Function     = 'Get-RemoteEventSummary'
            Status       = 'Failed'
            Data         = $null
            Error        = $_.Exception.Message
        }
    }
}
```

Two functions built. Both use the same skeleton. The only differences are the function name, the `Function` field value, and the script block content. The pattern is reusable.

> 🔧 **Try it**
>
> Run `Get-RemoteEventSummary -ComputerName 'System-1' -Credential $cred`. Then run it against a host that's reachable but does not have a System log entry matching the filter (rare, but a fresh image might). The function should still return a successful object, just with `$result.Data` containing fewer or zero items. `Status = 'Success'` means "the function ran without throwing," not "the result was non-empty." Internalize that distinction.

> ⭐ **Capstone connection**
>
> For the capstone, you will build `Get-RemoteServiceInventory`. Instead of `Get-WinEvent`, you will use `Get-Service`. The script block becomes `Get-Service | Select-Object Name, Status, StartType`. The wrapper does not change.

> ❓ **Check your understanding**
>
> Why does the function name change between practice and capstone, but the parameter block does not? What does that tell you about which parts of a function are interface and which are implementation?

5.4 - Build Function 3: Deploy Remote Support Files

Why this matters to an operator: Deployment is the third pattern your capstone requires, and it is structurally different from the Get-* pattern. It needs persistent sessions, file copy, verification, and explicit cleanup. The Get-* skeleton from 5.1 does not extend to deployment naturally, so this section gives you the second skeleton you need.

This function stages the same two files used later in the capstone: the Sysmon binary and the Sysmon configuration file. The goal here is to practice the deployment pattern cleanly without adding unnecessary structure.

There are five steps in the pattern: validate the local source paths, create the remote session, copy the files, verify that the files arrived in the expected destination, and close the session. That is enough for this lesson. It keeps the function readable and keeps the focus on the workflow students will actually need.

Function pattern

```
function Deploy-RemoteSupportFiles {
    [CmdletBinding()]
    param(
        [string]$ComputerName,
        [pscredential]$Credential,
        [string]$BinaryPath,
        [string]$ConfigPath
    )

    $Session = $null

    try {
        if (-not (Test-Path -Path $BinaryPath)) {
            throw "BinaryPath not found: $BinaryPath"
        }

        if (-not (Test-Path -Path $ConfigPath)) {
            throw "ConfigPath not found: $ConfigPath"
        }

        $Session = New-PSSession -ComputerName $ComputerName -Credential $Credential -
ErrorAction Stop

        Copy-Item -Path $BinaryPath -Destination 'C:\Windows\Temp\Sysmon64.exe' -ToSession
$Session -Force -ErrorAction Stop
        Copy-Item -Path $ConfigPath -Destination 'C:\Windows\Temp\sysmonconfig-export.xml'
-ToSession $Session -Force -ErrorAction Stop

        $Data = Invoke-Command -Session $Session -ScriptBlock {
            Get-Item 'C:\Windows\Temp\Sysmon64.exe', 'C:\Windows\Temp\sysmonconfig-
export.xml' |
                Select-Object Name, Length, LastWriteTime
        } -ErrorAction Stop

        [pscustomobject]@{
            ComputerName = $ComputerName
            Function     = 'Deploy-RemoteSupportFiles'
            Status       = 'Success'
            Data         = $Data
            Error        = $null
        }
    }
    catch {
        [pscustomobject]@{
            ComputerName = $ComputerName
            Function     = 'Deploy-RemoteSupportFiles'
            Status       = 'Failed'
```

```
            Data         = $null
            Error        = $_.Exception.Message
        }
    }
    finally {
        if ($Session) { Remove-PSSession $Session }
    }
}
```

What is different from the Get-* pattern

- An explicit `$Session` variable, declared as `$null` before the try block, so it can be referenced in the `finally` block
- A `finally` block that closes the session whether the function succeeded or failed
- Validation steps before any remote work begins (`Test-Path` on both source files)
- Two `Copy-Item` calls with `-ToSession` to push the files to the target
- A verification step (`Get-Item` inside an `Invoke-Command` on the same session) that confirms the files arrived

> 🔧 **Try it**
> Run the deployment function against System-1. Then run it again with a deliberately bad `BinaryPath` (something that does not exist locally). The function should fail cleanly, return a Status='Failed' object, and not leave a session open. Verify the second part with `Get-PSSession` : it should be empty after the failure.

> ⭐ **Capstone connection**
> For the capstone, you will build `Install-SysmonRemote` . The early steps (validate, create session, copy files) are the same. The capstone adds the install step after the copy succeeds. The deployment logic does not need to become more complicated first. Become comfortable with validation, session creation, file transfer, verification, and cleanup before adding installer execution.

> ⚠️ **The finally block is not optional**
> Without the `finally` block, a script that fails partway through leaves an orphaned session on the target. Multiply that across 50 test runs and you have 50 orphaned sessions. `finally` is what makes the function safe to run repeatedly. Get this habit in place now, before parallel execution multiplies the cost of forgetting it.

5.5 - Error Handling and Validation

Why this matters to an operator: Error handling and validation are what separate a script that runs in your test session from a script that runs reliably across multiple targets in a real workflow. The mechanics are simple. The discipline of applying them consistently is what matters.

Validate required files and inputs before launching remote actions. A clean function should fail early when the required local path, remote target, or input value is missing.

```
if (-not (Test-Path $ScriptPath)) {
    throw "Source script not found: $ScriptPath"
}
```

This pattern is short, but it gives you something important: the function fails on Ops, with a clear message about what went wrong, before any remote work begins. That is much easier to diagnose than a remote failure five steps in.

Consistent shapes for success and failure

Both success and failure paths should return the same object shape. This makes output easier to read, compare, export, and troubleshoot.

```
# Success output
[pscustomobject]@{
    ComputerName = $ComputerName
    Status       = 'Success'
    Error        = $null
}
```

```
# Failure output
[pscustomobject]@{
    ComputerName = $ComputerName
    Status       = 'Failed'
    Error        = $_.Exception.Message
}
```

Notice that both objects have exactly the same property names. The values differ. The shape does not. This is what makes the orchestrator's job easy in 5.7 and 5.8.

> ✅ **Practical standard**
> If a reviewer cannot tell the difference between your success and failure output shapes by

reading your code, your error handling needs work. The point is not that they look identical. The point is that they have the same property names so a downstream filter, sort, or export operation behaves predictably.

> 🔧 **Try it**
> Run any of your Chapter 5 functions against four targets where two are reachable and two are not. Pipe the result to `Where-Object Status -eq 'Failed'`. The filter should return exactly two rows. If it does not, your success and failure shapes are inconsistent and downstream code will not be able to rely on a uniform interface.

> ❓ **Check your understanding**
> Why does the failure object include `$_.Exception.Message` instead of just `$_`? What's the practical difference when you read the result later?

5.6 - Module Assembly

Why this matters to an operator: Once you have multiple functions, packaging them into a module is the only sensible way to share, reuse, and call them consistently. More importantly, jobs do not inherit the parent session's loaded functions. A module is what makes the parallel pattern work without copying function bodies into every job script block.

Once the practice functions work individually, the next step is to package them so they can be imported and reused as a single unit. That is the purpose of the module. In this course, the module is the container for your reusable tools, while the orchestrator is the script that decides when and where those tools run.

That distinction matters. A function belongs in the module because it represents a reusable capability. The orchestrator belongs in a separate script because its purpose is coordination: define the targets, gather the credential, import the module, call the functions, and collect the results. Keeping those roles separate makes the code easier to test, easier to update, and much easier to reason about once parallel jobs are introduced.

What the module is doing

- Storing the reusable functions in one file
- Exposing only the commands you intend to call from the outside
- Giving the orchestrator one import point instead of requiring function definitions to be copied into every script
- Giving each background job a clean way to load the same tools when parallel execution begins

> 💡 **Working model**
> Think of the module as the toolbox and the orchestrator as the work plan. The toolbox holds the tools. The work plan decides which tool runs, against which host, and in what order.

Example module structure

The example below shows the structure of a complete practice module. Notice that the file does not execute anything on its own. It only defines functions and exports them. That is exactly what you want from a reusable module.

```
# RemoteOpsTools.psm1

function Get-RemoteTaskInfo {
    # ... full function body from 5.2 ...
}

function Get-RemoteEventSummary {
    # ... full function body from 5.3 ...
}

function Deploy-RemoteSupportFiles {
    # ... full function body from 5.4 ...
}

Export-ModuleMember -Function Get-RemoteTaskInfo, Get-RemoteEventSummary, Deploy-
RemoteSupportFiles
```

Why the export list matters

The export list defines the module interface. When the module is imported, those are the commands you expect to be available in the session. This keeps the public surface of the module clear and prevents helper code or experimental functions from being exposed accidentally.

In a small training module the export list may feel optional, but it becomes more important as the function count grows. A clear export list also helps you confirm quickly whether the right module version is loaded.

Import and verify

Before moving on to the orchestrator, import the module directly and confirm that the expected commands appear. Then call one function to prove that the module is not only loading but also working.

```
$ModulePath = (Resolve-Path '.\RemoteOpsTools.psm1').Path

Import-Module $ModulePath -Force
Get-Command -Module RemoteOpsTools

$Credential = Get-Credential

Deploy-RemoteSupportFiles `
    -ComputerName 'System-1' `
    -Credential $Credential `
    -BinaryPath 'C:\Tools\Sysmon\Sysmon64.exe' `
    -ConfigPath 'C:\Tools\Sysmon\sysmonconfig-export.xml'
```

> 🔧 **Verification checkpoint**
> If the import succeeds but the commands are missing, inspect the function names and the `Export-ModuleMember` line first. If the commands appear but fail when called, test the function itself before moving to the orchestrator. Most module problems resolve at one of those two checks.

How this connects to jobs

The module is also what makes the job pattern manageable. A background job does not automatically inherit the functions that are already loaded in the parent session. Each job starts in its own runspace, so the module has to be imported again inside the job script block.

```
Start-Job -ArgumentList $ModulePath, $ComputerName, $Credential, $BinaryPath, $ConfigPath
-ScriptBlock {
    param($ModulePath, $ComputerName, $Credential, $BinaryPath, $ConfigPath)

    Import-Module $ModulePath -Force

    Deploy-RemoteSupportFiles `
        -ComputerName $ComputerName `
        -Credential $Credential `
        -BinaryPath $BinaryPath `
        -ConfigPath $ConfigPath
}
```

That is the practical reason the module section cannot be treated as just file organization. Without the module, the parallel pattern becomes clumsy very quickly. You would either have to redefine the functions inside each job or accept brittle execution flow. With the module in place, each job can load the same command set in a predictable way.

Module versus orchestrator

- **Module:** reusable functions only
- **Orchestrator:** target list, credential capture, import, execution order, and result collection
- **Job script block:** import the module again and run the exported functions for its assigned target

> ⭐ **Capstone bridge**
> In the capstone, the exact function bodies will change, but the relationship does not. The module remains the reusable command set. The orchestrator remains the script that coordinates how those commands are applied across multiple hosts. Naming changes (your capstone module is `CapstoneTools.psm1` and your orchestrator is `Invoke-Capstone.ps1`), but the structural separation does not.

? Check your understanding

If you import a module in your console session, then start a job, why does the job not see the module's functions? What's actually happening at the boundary?

5.7 - Orchestrator Pattern

Why this matters to an operator: An orchestrator that does too much is harder to maintain than two scripts that do less each. The orchestrator's job is coordination, not implementation. Get this division clean here and the capstone orchestrator practically writes itself.

The orchestrator is a coordinator. It imports the module, builds the target list, prompts for credentials, launches work, and collects results. It should be readable and short.

```
$Targets = 'System-1', 'System-2', 'System-3', 'Depot'
$Credential = Get-Credential

Import-Module '.\RemoteOpsTools.psm1' -Force

# Run against one target first to test
Get-RemoteTaskInfo -ComputerName 'System-1' -Credential $Credential
```

Five lines of meaningful code. That is what an orchestrator should look like at this stage. As you add parallel execution in 5.8, it will grow, but it should grow only as much as coordination requires.

> ✅ **Practical standard**
> Keep the orchestrator readable. If it is longer than 30-40 lines, you have probably put too much logic in it. The functions belong in the module. The script block content belongs in the functions. The orchestrator just decides who runs what against where.

> 🔧 **Try it**
> Save the orchestrator block above as `Run-RemoteOps.ps1`. Run it. Confirm one function call returns. Then add a second test call to `Get-RemoteEventSummary`. The script should grow by one line, not 30. That growth pattern is the test of a clean orchestrator.

> ⚠️ **Do not put function definitions in the orchestrator**
> The temptation, when the orchestrator is short, is to drop the function bodies directly in. Do not. Functions belong in the module. The orchestrator imports the module and calls them. This separation is what makes the parallel pattern in 5.8 possible at all.

> ❓ **Check your understanding**

If your orchestrator is 80 lines, where should the extra logic probably move to? Give two specific candidates from this chapter.

5.8 - Parallel Execution from Ops

Why this matters to an operator: Once functions work one target at a time, the orchestrator can fan out work in parallel. This is where Chapter 4's job patterns combine with this chapter's module pattern, and where the architecture of the capstone becomes visible. The mechanics are direct. The discipline is in keeping the orchestrator clean while it gets more capable.

Parallel execution from Ops is the moment Chapter 4 and Chapter 5 come together. The orchestrator launches one job per target, each job imports the module and calls one of the practice functions, and the results are collected after all jobs complete.

```
$Targets = 'System-1', 'System-2', 'System-3', 'Depot'
$Credential = Get-Credential
$ModulePath = (Resolve-Path '.\RemoteOpsTools.psm1').Path

Import-Module $ModulePath -Force

$Jobs = foreach ($Target in $Targets) {
    Start-Job -ArgumentList $Target, $Credential, $ModulePath -ScriptBlock {
        param($ComputerName, $Credential, $ModulePath)
        Import-Module $ModulePath -Force
        Get-RemoteTaskInfo -ComputerName $ComputerName -Credential $Credential
    }
}

$Results = $Jobs | Receive-Job -Wait -AutoRemoveJob
$Results | Format-Table -AutoSize
```

The pattern is the canonical pattern from Chapter 4.5: foreach target, start a job, pass parameters via `-ArgumentList`, receive all results when the jobs are done. The new piece is that the job script block imports the module and calls a function that you defined. That is what the module makes possible.

> 🔧 **Try it**
> Run the orchestrator above. Time it with `Measure-Command`. Then write a sequential version that runs the same function in a foreach loop without jobs, and time that. The parallel version should be roughly 4x faster, give or take. If it is not, check whether the parallel version is doing the same work or whether something inside the job is being serialized.

Counting data returned by wrapper objects

If a function returns a wrapper object with a `Data` property, count the items inside `Data`. Do not count the wrapper object itself.

```
# Correct: count the inventory data inside the wrapper object
ProcessCount = $ProcessResults.Data.Count
ServiceCount = $ServiceResults.Data.Count
```

⚠️ Watch the object shape

`$ProcessResults.Count` counts the wrapper object, not the inventory inside it. In PowerShell 7, a single scalar object reports a count of 1, even when the real inventory list contains many items. Use `$ProcessResults.Data.Count` to count the actual inventory.

⭐ Capstone connection

The capstone orchestrator looks almost exactly like the block above. The function name changes from `Get-RemoteTaskInfo` to `Get-RemoteProcessInventory` (or `Get-RemoteServiceInventory` or `Install-SysmonRemote` depending on which function the job runs). The module name changes from `RemoteOpsTools.psm1` to `CapstoneTools.psm1`. The structural pattern, including the foreach, the Start-Job, the Receive-Job pipeline, and the parameter passing, does not change.

❓ Check your understanding

Why must `Import-Module` appear inside the job script block, even though the orchestrator already imported the module before the foreach loop started?

5.9 - Troubleshooting and Cleanup

Why this matters to an operator: Repeated testing accumulates state. Sessions stay open. Jobs sit completed. Eventually your console becomes a mess and the symptoms look like script bugs. Cleanup is not an afterthought, it is part of the testing rhythm. Make it a habit.

When testing repeatedly, sessions and jobs accumulate. Make cleanup a habit.

```
Get-PSSession | Remove-PSSession
Get-Job | Remove-Job -Force
```

Diagnostic check table

When something does not work, run through this list before assuming the function logic is wrong. Most failures resolve at one of these checks.

Check	Command
Test credentials separately	`Invoke-Command -ComputerName System-1 -Credential $Cred -ScriptBlock { whoami }`
Confirm files exist on Ops	`Test-Path $BinaryPath`
Confirm remote host is reachable	`Test-WSMan System-1`
Read failure text from object	`$Results | Where-Object Status -eq 'Failed'`
Check orphaned sessions	`Get-PSSession`
Check orphaned jobs	`Get-Job`

> ⚠️ **Common mistake: accumulating stale sessions**
>
> If you test 10 times without cleanup, you may have 10 open sessions to each target. Run `Get-PSSession | Remove-PSSession` between test runs. The same applies to jobs: `Get-Job | Remove-Job -Force` is part of the test rhythm, not a one-time cleanup at the end of the day.

> 🔧 **Try it**
>
> After running the parallel orchestrator from 5.8 once, run `Get-PSSession` and `Get-Job`. Are they empty? If they are, the `-AutoRemoveJob` flag did its work. If not, you have leftover state.

> Now run the cleanup commands from this section. Then run the orchestrator again. Behavior should be identical between runs. If it is not, you have a state leak somewhere.

> ✅ **Build the cleanup habit before you need it**
> It is much easier to make cleanup automatic now, while you are testing three functions, than to retrofit cleanup discipline during the capstone when you are running everything across four targets in parallel. The habit transfers. The mess does not.

> ❓ **Check your understanding**
> Your orchestrator finished, but `Get-PSSession` shows two sessions still open. What does that tell you about which function is misbehaving, and what's the first thing you would check in that function's code?

5.10 - Chapter Review and Capstone Readiness

Why this matters to an operator: Section 5.10 is the readiness check before the formal exercise in 5.11 and the capstone in Chapter 6. If any of the capabilities below feel uncertain, now is the time to revisit the section that introduced them. The exercise that follows assumes these are automatic.

You are ready for the capstone when you can do the following with confidence:

- Build a remote query function using the pattern from 5.1
- Build a remote deployment function using the pattern from 5.4
- Return structured output from every function
- Handle per-host errors without crashing the workflow
- Assemble functions into a module and verify the import
- Write an orchestrator that coordinates work across all targets
- Execute work in parallel using `Start-Job`
- Clean up sessions and jobs as part of the testing rhythm

The capstone requires you to adapt, not copy

The capstone reuses the patterns from this chapter, but with different cmdlets and different deploy actions. The table below maps each capstone function to its Chapter 5 equivalent and names exactly what changes.

Capstone function	Chapter 5 equivalent	What changes
`Get-RemoteProcessInventory`	`Get-RemoteTaskInfo`	Script block: `Get-Process` instead of `Get-ScheduledTask`
`Get-RemoteServiceInventory`	`Get-RemoteEventSummary`	Script block: `Get-Service` instead of `Get-WinEvent`
`Install-SysmonRemote`	`Deploy-RemoteSupportFiles`	Two copies (binary+config), file verification, then add the install step

⚠ Practice files are not capstone deliverables

`RemoteOpsTools.psm1` is a practice module. It is not the capstone submission. Your capstone submission is `CapstoneTools.psm1`, a separately-named module containing the three required capstone functions. Do not submit your Chapter 5 practice work as your capstone deliverable. The function names, the module name, and the orchestrator name all change for the capstone.

> ✅ **The minimum bar before Chapter 6**
> If the diagnostic table in 5.9 is familiar, if the parallel orchestrator pattern in 5.8 is something you can write from memory, and if the difference between the module's job and the orchestrator's job is clear in your head, you are ready for the capstone. If any of those three is unclear, the gap is worth closing this evening before Friday.

> 💡 **Looking ahead to Chapter 6**
> Chapter 6 walks through the capstone build itself. It does not introduce new patterns. It applies the patterns from this chapter to the specific functions and module structure the capstone requires, then walks through the verification that distinguishes a passing submission from a failing one.

5.11 - Chapter Exercise: End-to-End Dry Run

Why this matters to an operator: This exercise is the final integration of Chapter 5. You import the module, run all three functions sequentially against one target, run all three in parallel across all four targets, verify the results, and clean up. By the end, you have empirical evidence that your toolkit works end-to-end. That evidence is what gives you confidence to attempt the capstone build on Friday.

> 💡 **Scenario**
> You have built `RemoteOpsTools.psm1` containing three functions: `Get-RemoteTaskInfo`, `Get-RemoteEventSummary`, and `Deploy-RemoteSupportFiles`. Before you attempt the capstone, you need to prove the full workflow works: module imports cleanly, functions return structured output, parallel execution gathers results from all four targets, failures are captured cleanly, and cleanup leaves the session in a known-good state.

Exercise Requirements

Build a script called `Run-RemoteOpsDryRun.ps1` that does the following:

1. **Setup:** Define the four targets, prompt for a credential, resolve the module path
2. **Import and verify:** Import the module with `-Force`, confirm with `Get-Command -Module RemoteOpsTools` that all three functions are available
3. **Sanity check:** Run `Get-RemoteTaskInfo` and `Get-RemoteEventSummary` against one target sequentially, confirm both return `Status='Success'`
4. **Parallel execution:** Use the canonical foreach + Start-Job pattern to run all three functions against all four targets simultaneously
5. **Collect and report:** Receive all results, format them as a table, filter for any failures with `Where-Object Status -eq 'Failed'`
6. **Cleanup:** Remove all sessions and all jobs

Exercise Skeleton: Run-RemoteOpsDryRun.ps1

```
# Chapter 5 Exercise: End-to-End Dry Run
# File: Run-RemoteOpsDryRun.ps1
# Goal: prove the full Chapter 5 workflow works end-to-end
#       before attempting the capstone build.

# --- Setup ---
$Targets = 'System-1', 'System-2', 'System-3', 'Depot'
$Credential = Get-Credential
$ModulePath = (Resolve-Path '.\RemoteOpsTools.psm1').Path

# --- Step 1: Import module and verify ---
# Import RemoteOpsTools.psm1 with -Force
# Confirm Get-Command -Module RemoteOpsTools shows all three functions

# --- Step 2: Sanity check one target sequentially ---
# Run Get-RemoteTaskInfo against System-1, confirm Status='Success'
# Run Get-RemoteEventSummary against System-1, confirm Status='Success'
# Skip Deploy-RemoteSupportFiles for this step

# --- Step 3: Run all three functions in parallel against all targets ---
$Jobs = foreach ($Target in $Targets) {
    Start-Job -ArgumentList $Target, $Credential, $ModulePath -ScriptBlock {
        param($ComputerName, $Credential, $ModulePath)
        Import-Module $ModulePath -Force

        # Call all three functions and emit each result
        Get-RemoteTaskInfo -ComputerName $ComputerName -Credential $Credential
        Get-RemoteEventSummary -ComputerName $ComputerName -Credential $Credential
        # Deploy-RemoteSupportFiles call goes here
    }
}

# --- Step 4: Collect results ---
$Results = $Jobs | Receive-Job -Wait -AutoRemoveJob

# --- Step 5: Verify and report ---
# Pipe $Results to Format-Table for a quick visual check
# Pipe $Results | Where-Object Status -eq 'Failed' to confirm no failures
# (or to see what failed and why)

# --- Step 6: Cleanup ---
Get-PSSession | Remove-PSSession
Get-Job | Remove-Job -Force
```

Success criteria

- The module imports without error and `Get-Command -Module RemoteOpsTools` shows all three functions
- Sequential test against one target returns `Status='Success'` for both Get-* functions
- Parallel run produces 12 result objects (4 targets × 3 functions)
- All result objects have the same property names: `ComputerName`, `Function`, `Status`, `Data`, `Error`
- `$Results | Where-Object Status -eq 'Failed'` returns either zero rows (clean run) or rows with the same shape as success rows (capturing what failed and why)
- After cleanup, both `Get-PSSession` and `Get-Job` return empty
- The script is re-runnable: running it twice in the same session produces consistent behavior

⚠️ Common gotchas on this exercise

Three failures consistently happen on the dry run:

- **Forgetting Import-Module inside the job script block.** Each job runs in its own runspace. Without the import inside the job, the function call fails with 'command not found.'
- **Argument order mismatch with `-ArgumentList`.** The order of values in `-ArgumentList` must match the order of `param()` inside the script block. Mixing them up gets credentials passed as computer names and produces confusing 'access denied' errors.
- **Skipping cleanup between test runs.** If your first run leaves orphaned sessions or jobs, your second run inherits them, and the result table fills with state from the first run that you forgot was there.

✅ Self-check before declaring this done

(1) Run the script. Does it complete without throwing?
(2) Does the result table show 12 rows with consistent property names?
(3) Run `Get-PSSession` and `Get-Job` after the script finishes. Both empty?
(4) Run the script a second time without restarting your session. Same behavior?
(5) Deliberately break one target (use a hostname that does not exist). Does the script still complete, with a Failed row instead of a thrown error?
If all five pass, you are ready for the capstone. If any fail, the failure points to which Chapter 5 concept needs more attention.

⚠️ Reminder before Chapter 6

This script is `Run-RemoteOpsDryRun.ps1` and uses `RemoteOpsTools.psm1`. The capstone deliverables are `Invoke-Capstone.ps1` and `CapstoneTools.psm1`. Different files, different function names, different cmdlets in the script blocks. Same patterns. Do not submit your dry run as your capstone.

Chapter 6: Capstone Build Standard

> 💡 **Chapter Purpose**
> This chapter is the build standard for the capstone. It does not introduce new patterns. It defines the standards your final module and orchestrator must meet, maps each standard to the rubric item it satisfies, and provides a validation and testing guide so you can self-verify your work before submission.

By the time you reach the capstone, you have already worked through the pieces that matter most: understanding returned objects, shaping output, writing functions, handling remote execution with PowerShell remoting, packaging functions in a module, and using jobs when work needs to scale across multiple systems. Chapter 6 shifts the focus from learning individual techniques to assembling them into one coherent solution that meets a specific assessment standard.

In this course, the student workstation is Ops. The remote targets are System-1, System-2, System-3, and Depot. The range is domain-based, so the workflow should assume normal domain credential use. That changes the environment, but it does not change the standard: the script still needs to be readable, dependable, and grounded in the PowerShell patterns practiced earlier.

What the capstone should demonstrate

A successful capstone shows more than simple command execution. It shows that the script author understands how to break a problem into functions, how to pass parameters instead of hard-coding choices, how to return structured output, how to handle remote work without turning the script into a tangle of one-off commands, and how to verify the result before declaring success.

Deliverable	What it should show
.psm1 tools module	Reusable functions with consistent parameters, structured output, and clean export behavior
.ps1 orchestrator	A readable control script that runs from Ops, prompts once for credentials, and coordinates the work
Two Get-* functions	Remote queries that return useful inventory data from the target systems
One deployment function	A controlled remote install of Sysmon using the supplied binary and configuration file

Chapter map

Section	Focus	Rubric coverage
6.1	Capstone requirements	R1, R2, R3, R4 (overview)
6.2	Module standard	R1, R3 (structure)
6.3	Required functions	R1, R3 (contracts)
6.4	Orchestrator standard	R1, R2 (control script)
6.5	Parallel execution standard	R2 (job dispatch)
6.6	Error handling standard	R3, R4 (deployment + try/catch)
6.7	Validation and testing guide	All four (verification)
6.8	Submission checklist	Final self-verification

✅ **Reading approach**

Treat this chapter as the build standard for the final exercise. Read it once for the big picture, then use each section as a reference while you implement and test your own module and orchestrator. The chapter does not contain a copy-paste solution. It contains the shape your solution must take and the rubric items it must satisfy.

⚠️ **This chapter does not introduce new techniques**

Every pattern referenced here was taught in Chapters 1 through 5. If a section here references a pattern that feels unfamiliar, the gap is in the earlier chapter, not in this one. Go back and shore up that pattern before continuing.

6.1 - Capstone Requirements

Why this matters to an operator: The capstone is intentionally narrow. That is a strength, not a limitation. The final build is supposed to prove that you can apply the course patterns to a realistic administrative task set without burying the solution under unnecessary complexity. Knowing what is required, and what is explicitly not required, lets you focus on the right things.

The solution runs from Ops and targets System-1, System-2, System-3, and Depot. Your deliverables are a PowerShell module and an orchestrator script that imports that module and uses its exported functions. The remote work should use PowerShell remoting. In this course, that means building around `New-PSSession`, `Invoke-Command`, and the patterns you practiced earlier.

Minimum required behavior

- Create a module that contains exactly the capstone functions you intend to use
- Include two remote inventory functions: `Get-RemoteProcessInventory` and `Get-RemoteServiceInventory`
- Include one remote deployment function: `Install-SysmonRemote`
- Use the supplied Sysmon files at `C:\Tools\Sysmon\Sysmon64.exe` and `C:\Tools\Sysmon\sysmonconfig-export.xml` in your examples and testing workflow
- Return structured output rather than writing status text with `Write-Host`
- Use one credential prompt at the orchestrator level and pass that credential into the work that follows
- Support one target during early testing and all four targets during final execution

> 💡 **Practical standard**
> The capstone does not need a self-healing framework, persistent credential vaulting, TrustedHosts reconfiguration logic, or multiple fallback remoting methods. A clean solution that reliably uses the taught pattern is stronger than a sprawling solution that tries to solve every possible environment problem.

Expected execution model

The orchestration flow should be easy to explain in plain language: prompt for credentials once, import the module, run the required functions against the target systems, collect the results, and present those results in a way that makes success or failure obvious. If another student or instructor cannot follow the flow quickly by reading the script top to bottom, the design is probably carrying too much overhead.

```
$Targets = 'System-1', 'System-2', 'System-3', 'Depot'
$ModulePath = (Resolve-Path '.\CapstoneTools.psm1').Path
$Credential = Get-Credential
Import-Module $ModulePath -Force
Get-RemoteProcessInventory -ComputerName 'System-1' -Credential $Credential
Get-RemoteServiceInventory -ComputerName 'System-1' -Credential $Credential
Install-SysmonRemote -ComputerName 'System-1' -Credential $Credential `
    -BinaryPath 'C:\Tools\Sysmon\Sysmon64.exe' `
    -ConfigPath 'C:\Tools\Sysmon\sysmonconfig-export.xml'
```

That example is not the finished capstone. It shows the operating model. Before parallel execution or full orchestration enters the picture, each individual function should work cleanly against one target. That staging matters because it keeps troubleshooting focused. When something fails, you want to know whether the problem is in the function itself or in the orchestration wrapped around it.

What strong submissions usually have in common

Characteristic	What it looks like in practice
Consistent parameters	Each function accepts at least `-ComputerName` and `-Credential` in a predictable way
Structured return values	Each function returns objects that identify the target and the result clearly
Readable control flow	The orchestrator is short enough to understand without hunting through nested logic
Tidy cleanup	Sessions, jobs, and temporary artifacts are cleaned up deliberately
Incremental testing	The student proves one target first, then expands to all required targets

The rubric you are graded against

The capstone is evaluated against four rubric items. Each item traces to one CTSSB skill. All four items must be satisfied for the capstone to be assessed as demonstrating the required skill. The rubric below is verbatim from the assessment scope contract.

ID	Skill	Criterion	Pass condition
R1	S1	Module contains two or more Get-* functions that query remote systems	Each function accepts a -ComputerName parameter, returns a PSCustomObject, and successfully retrieves data from at least two distinct remote hosts.

ID	Skill	Criterion	Pass condition
R2	S2	Orchestrator executes module functions in parallel	Jobs are dispatched via Start-Job or ForEach-Object -Parallel. Execution is demonstrably concurrent.
R3	S3	Module contains a function that deploys Sysmon to a remote system	The Sysmon binary and configuration file are transferred to the target. The Sysmon service is installed and running on the target upon completion. State is verifiable via Get-Service executed against the target.
R4	S4	Script handles errors without terminating execution	try/catch blocks enclose remote operations and deployment logic. Script execution completes regardless of per-target failures. Output differentiates successful targets from failed targets.

✅ **How to use the rubric while you build**

Each subsequent section in this chapter (6.2 through 6.7) addresses one or more of the four rubric items. The chapter map at the front of Chapter 6 shows which items each section covers. Use the rubric as a checklist while you build, not just at submission time. If you can name which rubric item a piece of your code satisfies, that piece is probably in good shape. If you cannot, it may be doing the wrong job.

⚠️ **What to avoid**

Do not treat the capstone as a contest to write the most complicated script. Nested remoting patterns, mixed remoting technologies in the same function, or deep layers of recovery logic usually make the solution harder to validate and harder to explain. The rubric does not reward complexity. It rewards clear adherence to the four pass conditions.

❓ **Check your understanding**

Read the rubric above. For each rubric item, name one Chapter 5 section where the underlying pattern was taught. If you cannot, that's the gap to close before you start building.

6.2 - Module Standard

Why this matters to an operator: The module is the part of your capstone that holds reusable capability. Get the structure right and the rest of the build follows naturally. The standard is straightforward: function definitions, validation, structured return, explicit export. Nothing else.

The module is the reusable part of the capstone. It should contain the functions that do the real work while the orchestrator stays focused on coordination. That separation matters because it makes testing easier. You can import the module and test one function at a time before you ever run the full `.ps1` file.

A good module reads like a small toolset. The exported functions are clear, the parameter sets are predictable, and nothing important happens merely because the module was imported. Importing a module should load capability. It should not silently launch work.

What belongs in the module

- Function definitions
- Parameter validation that supports the function contract
- Structured return objects
- Session creation and cleanup inside the functions that need it
- An explicit export statement for the functions the orchestrator should call

What should stay out of the module

- Interactive prompts that fire on import
- Long blocks of one-time testing code
- Hard-coded credentials
- Ad hoc transcript text written with `Write-Host` as the primary result
- Top-level commands that execute automatically when the module is imported

Module structural skeleton

The structure below shows the module-level shape. Every function must have its body filled in by you. The skeleton shows the parameter contracts (which are required by R1) and the export statement (which is required to make the functions visible after import).

```
function Get-RemoteProcessInventory {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string]$ComputerName,
        [Parameter(Mandatory)]
        [pscredential]$Credential
    )
    # Function body
}

function Get-RemoteServiceInventory {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string]$ComputerName,
        [Parameter(Mandatory)]
        [pscredential]$Credential
    )
    # Function body
}

function Install-SysmonRemote {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string]$ComputerName,
        [Parameter(Mandatory)]
        [pscredential]$Credential,
        [Parameter(Mandatory)]
        [string]$BinaryPath,
        [Parameter(Mandatory)]
        [string]$ConfigPath
    )
    # Function body
}

Export-ModuleMember -Function Get-RemoteProcessInventory, Get-RemoteServiceInventory,
Install-SysmonRemote
```

Why the parameter pattern matters

Keeping the parameter pattern consistent reduces friction everywhere else in the build. When each function accepts `-ComputerName` and `-Credential` in the same way, the orchestrator can call them predictably and you can troubleshoot them predictably. In practice, consistency is one of the fastest ways to keep a capstone from becoming messy.

Design choice	Recommended standard	Reason
Function names	Use verb-noun names such as `Get-RemoteProcessInventory`	The purpose is obvious before the function is opened
Parameter names	Keep shared parameters identical across functions	The orchestrator can call them without special-case logic
Return type	Return PowerShell objects	Objects can be filtered, counted, exported, and tested
Export behavior	Use `Export-ModuleMember`	Only the intended public functions are exposed

> 💡 **Module mindset**
> Think of the module as the part you would want to keep after the course ends. If the orchestrator were deleted, the module should still contain functions that make sense and can be tested on their own.

Import and verify before building the orchestrator

```
Import-Module '.\CapstoneTools.psm1' -Force
Get-Command -Module CapstoneTools
```

If the import does not expose the functions you expect, stop there and fix the module first. Troubleshooting a failed orchestrator when the module import is already wrong wastes time and hides the real issue.

> ⚠️ **Common source of confusion**
> A function existing in the file is not the same thing as the function being available after import. Missing export statements, syntax errors, or accidental top-level code can make a module behave very differently from what the file appears to contain.

> ✅ **Rubric coverage**
> This section's standards address **R1** (module contains two or more Get-* functions, each accepting -ComputerName) and **R3** (module contains a deployment function). The function bodies are your work. The parameter contracts and export statement are the standard you must meet.

6.3 - Required Functions

Why this matters to an operator: The capstone requires three functions, but the deeper requirement is that each one should have a clear job and a clear output contract. A reader should be able to identify what a function is supposed to do by its name, its parameter block, and the object shape it returns. This section names the contracts. The implementations are yours.

1) Get-RemoteProcessInventory

This function should connect to the remote system, retrieve process information, and return inventory data as objects. The point is not to dump every property available from `Get-Process`. The point is to choose a useful set of properties and return them consistently.

```
Get-RemoteProcessInventory -ComputerName 'System-1' -Credential $Credential |
Select-Object ComputerName, Name, Id, CPU
```

> 💡 **Recommended mindset**
> Choose properties that help you recognize what is running and where the record came from. Including the source system in the returned object is especially important once you move to multiple targets, because it is the only way to tell which result came from which host after the orchestrator merges them.

2) Get-RemoteServiceInventory

This function should connect to the remote system, retrieve service information, and return objects that make service state easy to review. Again, the goal is not maximum property volume. The goal is useful, readable inventory data.

```
Get-RemoteServiceInventory -ComputerName 'System-1' -Credential $Credential |
Select-Object ComputerName, Name, DisplayName, Status, StartType
```

3) Install-SysmonRemote

This function is the deployment function. It should validate the provided file paths, create a remote session, copy the Sysmon binary and configuration file to the target, execute the installer, verify the install, and return a summary object. The capstone path examples should remain aligned to the course standard: `C:\Tools\Sysmon\Sysmon64.exe` and `C:\Tools\Sysmon\sysmonconfig-export.xml`.

```
Install-SysmonRemote -ComputerName 'System-1' -Credential $Credential `
    -BinaryPath 'C:\Tools\Sysmon\Sysmon64.exe' `
    -ConfigPath 'C:\Tools\Sysmon\sysmonconfig-export.xml'
```

Think in function contracts

One of the easiest ways to strengthen a capstone solution is to define what success and failure should look like before writing the full function body. That keeps the function from drifting into vague behavior.

Function	Should accept	Should return
`Get-RemoteProcessInventory`	`-ComputerName`, `-Credential`	One or more objects describing process inventory from the target system
`Get-RemoteServiceInventory`	`-ComputerName`, `-Credential`	One or more objects describing service inventory from the target system
`Install-SysmonRemote`	`-ComputerName`, `-Credential`, `-BinaryPath`, `-ConfigPath`	A summary object that clearly indicates success or failure for the installation attempt

The deployment function's return is described as "a summary object that clearly indicates success or failure." That is the contract. The exact property names and values you choose are part of your design. What matters is that a reader (and the rubric) can determine the install status from the object and that success and failure paths produce the same shape.

> ✅ **Strong capstone habit**
> Keep each function responsible for one category of work. Inventory functions should gather and return data. The deployment function should perform the install workflow. Once functions begin mixing unrelated tasks, they become harder to test and harder to trust.

> ⚠️ **Do not overbuild the deployment function**
> The deployment function does not need to manage every possible environmental exception. It should handle the taught workflow cleanly, return a usable result, and clean up after itself. That is the standard to aim for. The rubric requires R3 (Sysmon installed and verifiable via `Get-Service`) and R4 (try/catch around remote ops). It does not require a self-healing installer.

> ✅ **Rubric coverage**
> This section's contracts address **R1** (Get-* functions return PSCustomObject, accept -

ComputerName) and **R3** (deployment function transfers files and installs Sysmon). The verification requirement under R3 is operational: `Get-Service Sysmon64` against the target should return the service in a Running state after your function completes.

? Check your understanding

The contract table says the deployment function should return 'a summary object that clearly indicates success or failure.' What property names would you choose for that object? Justify each name in one sentence.

6.4 - Orchestrator Standard

Why this matters to an operator: An orchestrator that does too much is harder to maintain than a script that does less and does it cleanly. The orchestrator's job is coordination, not implementation. Get this division clean and the testing in 6.7 becomes straightforward.

The orchestrator is the control script. It should not contain the deepest technical work. Its job is to set the run conditions, prompt for credentials, import the module, call the exported functions in a sensible sequence, and present the results.

A good orchestrator reads from top to bottom without surprises. You should be able to identify the input parameters, the module path, the target systems, the credential collection point, and the work sequence without scrolling through dense nested logic.

Recommended responsibilities

- Accept the target systems and module path as parameters
- Prompt once for credentials with `Get-Credential`
- Resolve and import the module
- Run the required module functions
- Collect and display results
- Clean up background jobs if the script uses parallel execution

Recommended parameter block

```
[CmdletBinding()]
param(
    [string[]]$ComputerName = @('System-1', 'System-2', 'System-3', 'Depot'),
    [string]$ModulePath = (Join-Path $PSScriptRoot 'CapstoneTools.psm1')
)
$Credential = Get-Credential
$ResolvedModulePath = (Resolve-Path $ModulePath).Path
Import-Module $ResolvedModulePath -Force
# Orchestration logic follows
```

Two parameter names matter for testing: `-ComputerName` (the list of targets) and `-ModulePath` (the path to your module). The validation guide in 6.7 invokes the orchestrator using these parameter names. Use them in your own orchestrator so the test commands work without modification.

> 💡 **Why the credential prompt belongs here**
> The orchestrator is the right place to collect credentials because it sits at the start of the run

and can pass that credential object into the rest of the work. That keeps the module reusable and avoids repeated prompts. It also avoids a common failure pattern: trying to prompt for credentials from inside background jobs or nested remote commands.

A readable sequential sequence

```
$Targets = $ComputerName
foreach ($Target in $Targets) {
    Get-RemoteProcessInventory -ComputerName $Target -Credential $Credential
    Get-RemoteServiceInventory -ComputerName $Target -Credential $Credential
    Install-SysmonRemote -ComputerName $Target -Credential $Credential `
        -BinaryPath 'C:\Tools\Sysmon\Sysmon64.exe' `
        -ConfigPath 'C:\Tools\Sysmon\sysmonconfig-export.xml'
}
```

That single-threaded version is worth building first even if your final standard uses jobs. It is easier to read, easier to debug, and easier to validate. Once the sequential version works, you can change the control structure to parallel execution without changing the function contracts.

Part of script	Keep it focused on
Parameter block	Inputs to the run
Credential collection	One prompt at the start
Module import	Loading the reusable tools
Execution block	Calling the required functions in a clear order
Result handling	Presenting what happened without hiding errors

💡 **A useful design test**
If you can explain the orchestrator in five or six short sentences, the structure is probably in good shape. If the explanation turns into a tour of exceptions, special cases, and workarounds, the script likely needs to be simplified.

⚠️ **Keep one-time commands out of the final script**
Commands used only for ad hoc troubleshooting, such as exploratory remoting tests, should be used during validation but should not stay embedded in the final orchestrator unless they are part of the intended execution flow.

✅ **Rubric coverage**
This section's standards address **R1** (orchestrator calls Get-* functions against multiple hosts) and **R2** (orchestrator coordinates parallel execution, taught in 6.5). The sequential

> block above is a stepping stone, not the final form. Your submitted orchestrator must dispatch work in parallel to satisfy R2.

? Check your understanding

Why does the parameter block use `$PSScriptRoot` as the default for `$ModulePath` ? What does this default behavior do for someone who runs your script from a different working directory?

6.5 - Parallel Execution Standard

Why this matters to an operator: Parallel execution is what turns a sequential walk through four targets into a single fanned-out run. The mechanics are direct. The discipline is in keeping the orchestrator readable while it gets more capable. The skeleton below shows the shape. The implementation is your work.

Parallel execution is useful here because the capstone targets multiple remote systems. The gain is not just speed. Parallel execution also gives you a cleaner model for "do the same work on each system" when the functions are already built and tested.

In this course, the practical way to introduce parallel work is with background jobs. Each job receives the module path, the target system name, and the credential object. Inside the job, the module is imported again because each background job starts in a separate session state.

> 💡 **Important detail**
> If the orchestrator uses jobs, do not call `Get-Credential` inside the job. Prompt once before the jobs are created, then pass the credential object into each job with `-ArgumentList`. This was taught in Chapter 4.4 and Chapter 5.8.

Parallel pattern skeleton

The skeleton below shows the structure your parallel block must take. The pattern from Chapter 5.8 is the same. The work in the comments is yours to fill in.

```
$Targets = 'System-1', 'System-2', 'System-3', 'Depot'
$Credential = Get-Credential
$ModulePath = (Resolve-Path '.\CapstoneTools.psm1').Path

$Jobs = foreach ($ComputerName in $Targets) {
    Start-Job -Name $ComputerName -ArgumentList # pass module path, computer name,
credential, file paths `
        -ScriptBlock {
        param(# matching parameter names in the same order)

        # 1. Import the module inside the job
        #    (the parent session's import does NOT carry into the job)

        # 2. Call the three module functions:
        #    Get-RemoteProcessInventory
        #    Get-RemoteServiceInventory
        #    Install-SysmonRemote (with the binary and config paths)

        # 3. Optional: emit a per-target summary object that
        #    consolidates the three function results into one row
    }
}

# Wait, receive, remove (or Receive-Job -Wait -AutoRemoveJob)
$null = $Jobs | Wait-Job
$Results = $Jobs | Receive-Job
$Jobs | Remove-Job
$Results
```

Why this pattern works

Element	Reason it matters
`Start-Job`	Creates one unit of work per target system
`-ArgumentList`	Passes the values the job needs without relying on outer scope
Import-Module inside the job	Each job has its own runspace and must load the module for itself
`Wait-Job` and `Receive-Job`	Keeps collection of results orderly
`Remove-Job`	Cleans up after the run

The job does not attempt to become an orchestrator inside the orchestrator. It imports the module, calls the required functions, and (optionally) returns a small summary object. That keeps the result compact and keeps the parallel block readable.

Build in this order

1. Make each function work against one target
2. Make the orchestrator work sequentially against one target
3. Expand the sequential run to all required targets
4. Introduce background jobs only after the previous steps are clean
5. Verify that the job results still return the information you expect

Skipping a step in this list is the most common cause of capstone time loss. Sequential success is what tells you the functions work. Parallel success is what tells you the orchestration around them works. Trying to validate both at once makes diagnosing failures harder, not easier.

> ⚠️ **Where students usually lose time**
>
> Most job-related problems are not job problems. They are missing module imports inside the job, missing arguments in `-ArgumentList`, argument order mismatches between `-ArgumentList` and `param()`, or attempts to use interactive prompts from inside the job. Check those four things before assuming the job mechanism itself is broken.

> ✅ **Reasonable standard**
>
> A compact job wrapper around already-tested functions is enough. The capstone does not need nested jobs, threaded runspace frameworks, or a custom job-monitoring system. The rubric requires demonstrably concurrent execution. It does not require a custom orchestration framework.

> ✅ **Rubric coverage**
>
> This section's standards address **R2** (orchestrator dispatches jobs via `Start-Job` or `ForEach-Object -Parallel`, execution is demonstrably concurrent). Your final orchestrator must implement this pattern (or a structurally equivalent `ForEach-Object -Parallel` variant) to satisfy R2.

> ❓ **Check your understanding**
>
> The skeleton's `Start-Job` line shows `-ArgumentList` with a comment placeholder. List the actual values you would pass, and the order they would appear. Then write the matching `param()` line. Does the order match?

6.6 - Error Handling Standard

Why this matters to an operator: Error handling should make the capstone easier to trust, not harder to read. The clean standard is to use one clear try/catch/finally pattern around the work that can fail, convert non-terminating problems into terminating ones where needed, and return a result that explains what happened. The skeleton below shows the shape. The implementation choices are yours.

This is one of the places where restraint matters. Deep nesting is rarely a sign of strength. In most capstone functions, a single outer try block is enough when the steps inside it use explicit error behavior and the cleanup is handled in `finally`.

Skeleton: Install-SysmonRemote with error handling

The skeleton below shows the try/catch/finally shape required to satisfy R4 (try/catch around remote operations) while also satisfying R3 (Sysmon transferred, installed, verifiable). The body of each step is your work. The structure is the standard you must meet.

```powershell
function Install-SysmonRemote {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string]$ComputerName,
        [Parameter(Mandatory)]
        [pscredential]$Credential,
        [Parameter(Mandatory)]
        [string]$BinaryPath,
        [Parameter(Mandatory)]
        [string]$ConfigPath
    )

    $Session = $null

    try {
        # 1. Validate local paths exist (Test-Path on $BinaryPath, $ConfigPath)
        #    If either is missing, throw with a clear message

        # 2. Open a remote session
        #    Assign result to $Session so it's visible in finally

        # 3. Copy the binary and config to the target
        #    Use Copy-Item with -ToSession

        # 4. Execute the installer on the target
        #    The taught approach uses Invoke-Command -Session

        # 5. Verify the install succeeded
        #    Get-Service against the target should show Sysmon running

        # 6. Return a success object
        #    Shape: ComputerName, Function, Status='Success', and a details field
    }
    catch {
        # Return a failure object with the SAME shape as success
        # Use $_.Exception.Message for the failure detail
    }
    finally {
        # If a session was opened, close it
        if ($Session) { Remove-PSSession $Session }
    }
}
```

What this pattern gives you

Feature	Benefit
Path checks before remoting	Simple local problems are caught before a session is opened
`-ErrorAction Stop`	Cmdlets that would otherwise continue can be handled in the catch block
Single catch block	Failure handling stays readable and consistent
`finally` cleanup	The session is removed whether the function succeeds or fails
Structured failure object	The caller can still work with the result programmatically

When nested try blocks are usually unnecessary

Sometimes students start nesting try blocks around every individual step. That usually makes the function harder to read without improving the result. In a capstone of this size, nested try blocks are only justified when different failures truly need different recovery actions. If the handling is the same, return a failure object and clean up, one outer structure is usually the better design.

> 💡 **Helpful testing habit**
> During validation, force a known failure once. Use a bad file path or an unavailable target and confirm that the function returns a clean failure object instead of breaking the entire script flow in a confusing way. Phase 7 of the testing guide in 6.7 walks through this exact test.

> ⚠️ **Avoid these patterns**
>
> - Using `Write-Host` as the only error output
> - Returning plain strings for failure when success returns objects
> - Leaving sessions open when an exception occurs
> - Catching an error and then hiding the useful message
> - Using a bare `catch {}` block that swallows the exception entirely

> ✅ **Rubric coverage**
> This section's standards address **R3** (Sysmon transferred, installed, and verifiable on the target) and **R4** (try/catch around remote operations, output differentiates successful targets from failed targets). The verification required by R3 happens inside the try block before the success object is returned. The error differentiation required by R4 happens through the consistent shape of success and failure objects.

> ❓ **Check your understanding**

Why must `$Session` be declared as `$null` before the try block, instead of inside it? What goes wrong if the declaration is moved into try?

6.7 - Validation and Testing Guide

Why this matters to an operator: Validation should move from simple to complex. The fastest way to burn time in the capstone is to test everything at once. The phased approach below moves you from quick environment checks to full orchestration, with failure injection at the end. Each phase answers one question. Failures isolate to one layer at a time.

The fastest way to burn time in the capstone is to test everything at once. A better sequence is to verify the environment, test the module import, test remoting, test each function against one target, and only then run the orchestrator across multiple targets.

Phase 1: Verify the local prerequisites on Ops

```
Test-Path '.\CapstoneTools.psm1'
Test-Path '.\Invoke-Capstone.ps1'
Test-Path 'C:\Tools\Sysmon\Sysmon64.exe'
Test-Path 'C:\Tools\Sysmon\sysmonconfig-export.xml'
```

At this phase, you are only proving that the expected files are present where the script expects them to be. If the file paths are wrong, fix that first.

Phase 2: Verify the module import

```
Import-Module '.\CapstoneTools.psm1' -Force
Get-Command -Module CapstoneTools
```

The command list should show the three capstone functions. If it does not, stop and correct the module before moving on.

Phase 3: Verify remoting against one target

```
$Credential = Get-Credential
Invoke-Command -ComputerName 'System-1' -Credential $Credential -ScriptBlock {
    hostname
}
```

> 💡 **Reason for this step**
> This confirms that the environment can support the function calls that depend on remoting. It also separates basic remoting problems from function logic problems.

Phase 4: Unit-test each function

```
Get-RemoteProcessInventory -ComputerName 'System-1' -Credential $Credential
Get-RemoteServiceInventory -ComputerName 'System-1' -Credential $Credential
Install-SysmonRemote -ComputerName 'System-1' -Credential $Credential `
    -BinaryPath 'C:\Tools\Sysmon\Sysmon64.exe' `
    -ConfigPath 'C:\Tools\Sysmon\sysmonconfig-export.xml'
```

Each function should be proven individually. If one of them fails, fix that function first rather than masking the failure by moving into the orchestrator.

Phase 5: Run the orchestrator against one target

```
'.\Invoke-Capstone.ps1' -ComputerName 'System-1' `
    -ModulePath (Resolve-Path '.\CapstoneTools.psm1').Path
```

This phase uses the parameter names defined in 6.4. If your orchestrator uses different parameter names, adjust this command to match.

Phase 6: Expand to all required targets

```
'.\Invoke-Capstone.ps1' -ComputerName 'System-1', 'System-2', 'System-3', 'Depot' `
    -ModulePath (Resolve-Path '.\CapstoneTools.psm1').Path
```

Phase 7: Perform one deliberate failure test

```
Install-SysmonRemote -ComputerName 'System-1' -Credential $Credential `
    -BinaryPath 'C:\Tools\Sysmon\Missing.exe' `
    -ConfigPath 'C:\Tools\Sysmon\sysmonconfig-export.xml'
```

That last test is valuable because it proves the function's failure handling. A clean failure result is part of a good solution. Without this test, you cannot know whether your error handling actually fires or whether the catch block is silently broken.

R3 verification: confirm Sysmon is running on the target

The R3 rubric item explicitly requires that Sysmon state be verifiable via `Get-Service` executed against the target after your deployment function runs. Run this against any target where `Install-SysmonRemote` reported success:

```
Invoke-Command -ComputerName 'System-1' -Credential $Credential -ScriptBlock {
    Get-Service -Name 'Sysmon64'
}
```

The result should show `Sysmon64` (or whatever the installed Sysmon service name is on your range image) in a Running state. If your deployment function reports success but `Get-Service` cannot find the service, your install verification step is missing or wrong.

Test summary

Test level	Question being answered
File path checks	Are the required inputs present?
Module import	Are the exported functions available?
Basic remoting	Can Ops reach the target with the provided credential?
Function tests	Does each tool work on its own?
One-target orchestration	Does the control script call the tools correctly?
Full orchestration	Does the final workflow scale to all required systems?
Failure injection	Does error handling produce a clean failure result?

✅ **Efficient troubleshooting rule**
When a higher-level test fails, move back down one layer. Do not keep testing the orchestrator when the individual function has already shown you where the issue lives. The whole point of the phased approach is that each phase narrows the problem space.

✅ **Rubric coverage**
This section provides the verification path for all four rubric items. **R1** verified at Phase 4 (each Get-* function returns PSCustomObject). **R2** verified at Phase 6 (parallel execution across all four targets). **R3** verified by the `Get-Service Sysmon64` check after Install-SysmonRemote completes. **R4** verified at Phase 7 (failure injection produces a clean failure object, not a thrown exception).

❓ **Check your understanding**

Phase 7 forces a deliberate failure. What specific behavior are you watching for during that test? What should happen, and what would indicate that your error handling is broken?

6.8 - Submission Checklist

Why this matters to an operator: The last step before submission is not writing more code. It is confirming that the code you already wrote meets the standard you intended to meet. A short checklist is useful here because it prevents last-minute drift, and the checklist below is aligned to the actual rubric language so you can self-verify against what is being graded.

Final review checklist (rubric-aligned)

Each item below maps to one or more rubric items. Run through this checklist before you submit.

- The module imports cleanly with `Import-Module .\CapstoneTools.psm1 -Force` *(R1)*
- Only the intended capstone functions are exported *(R1)*
- The required functions are present: `Get-RemoteProcessInventory`, `Get-RemoteServiceInventory`, and `Install-SysmonRemote` *(R1, R3)*
- Each Get-* function accepts `-ComputerName` and returns a PSCustomObject *(R1)*
- Each Get-* function successfully retrieves data from at least two distinct remote hosts *(R1)*
- The orchestrator dispatches work in parallel via `Start-Job` or `ForEach-Object -Parallel` *(R2)*
- Parallel execution is demonstrably concurrent (the run is faster than a sequential equivalent against the same targets) *(R2)*
- `Install-SysmonRemote` transfers both the binary and the configuration file to the target *(R3)*
- After `Install-SysmonRemote` reports success, `Get-Service` executed against the target shows Sysmon running *(R3)*
- `try/catch` blocks enclose remote operations and deployment logic *(R4)*
- Script execution completes regardless of per-target failures *(R4)*
- Output differentiates successful targets from failed targets *(R4)*
- The orchestrator prompts once for credentials with `Get-Credential`
- Sessions and jobs are cleaned up
- The script paths for the Sysmon binary and configuration file match the course standard

A final command sequence

```
Import-Module '.\CapstoneTools.psm1' -Force
Get-Command -Module CapstoneTools
'.\Invoke-Capstone.ps1' -ComputerName 'System-1', 'System-2', 'System-3', 'Depot' `
    -ModulePath (Resolve-Path '.\CapstoneTools.psm1').Path
```

What to look for in the final output

Area	What you want to see
Inventory results	PSCustomObjects that clearly identify the source system and the returned data (R1)
Deployment results	Success or failure status with a useful details field; Sysmon service running on every target reported as Success (R3)
Concurrency	Demonstrably concurrent execution via Start-Job or ForEach-Object -Parallel (R2)
Failure differentiation	Output makes it obvious which targets succeeded and which failed; script does not abort on per-target failure (R4)
Overall readability	Output that can be explained quickly without digging through raw console noise
Script behavior	A predictable run with no hidden prompts and no leftover jobs or sessions

> 💡 **Professional habit**
> A good submission is one you can defend line by line. If an instructor points at a function, a parameter, or a block in the orchestrator, you should be able to explain why it exists and what it returns. The rubric does not require you to defend your code verbally, but if you cannot, that is usually a sign that part of the code is not really yours yet.

> ✅ **Rubric coverage**
> This checklist is the final self-verification against all four rubric items. Each checkbox traces to an R1, R2, R3, or R4 pass condition. If every item is checked, your submission has the structural elements the rubric requires. The remaining quality of the work is in the implementation choices you made.

That is the real finish line for the capstone. The code should run, but it should also be understandable, testable, and built from patterns you can reuse in later scripting work.

> ❓ **Final check**
> Read the rubric in 6.1 one more time. For each of the four rubric items (R1, R2, R3, R4), name which file in your submission satisfies it and which specific code block within that file proves the pass condition. If any item is hard to point at, that's the area to revisit before submission.